Web Fundamentals and UX Design - Complete Study Notes

Course Code: CS1307 | **Semester:** 1 | **RV University**

Course Overview

This comprehensive guide covers all essential topics for the Web Fundamentals and UX Design course. These notes are designed to serve as your complete study resource for mastering web development fundamentals, CSS styling, JavaScript programming, and responsive design principles.

Learning Objectives

By mastering this material, you will be able to:

- **CO1:** Build static web pages using HTML (HyperText Markup Language) [1] [2]
- **CO2:** Apply CSS (Cascading Style Sheets) to create visually appealing designs [1] [2]
- **CO3:** Develop interactive and dynamic web pages using JavaScript with event handling and DOM manipulation [1] [2]
- **CO4:** Apply Bootstrap to design and develop responsive web applications [1] [2]

Module 1: Internet Fundamentals and HTML

Unit 1.1: Internet Architecture and Protocols

The Internet: Fundamental Concepts

Definition: The Internet is the largest network in the world that connects hundreds of thousands of individual networks globally. It enables the electronic movement of ideas and information through cyberspace [2].

Core Functions:

- Send and receive email messages
- Upload and download files between computers
- Access web resources (surfing the web) [2]

Historical Evolution of the Internet

Timeline of Major Developments

Era	Development	Significance
1950s-1960s	ARPA begins ARPANET work	Foundation of internet infrastructure [2]
1969	ARPANET becomes operational	First message "LOGIN" sent between universities [2]
1970s	TCP/IP developed by Vint Cerf and Bob Kahn	Foundational technology enabling network communication [2]
1980s	DNS introduced, TCP/IP standardized	Human-readable domain names enabled [2]

Era	Development	Significance
1990s	World Wide Web invented by Tim Berners-Lee	Web browsers make internet accessible to public [2]
2000s	Broadband and social media emergence	Facebook, Twitter, YouTube reshape communication [2]
2010s	Mobile internet and cloud computing	Smartphones and IoT devices transform usage [2]
2020s	5G networks, AI, blockchain integration	Faster speeds, emerging technologies reshape internet services [2]

IP Addressing Systems

IP Address: A numerical label assigned to each device connected to a network using the Internet Protocol [2].

IPv4 vs IPv6 Comparison

Aspect	IPv4	IPv6
Structure	32-bit decimal address	128-bit hexadecimal address [2]
Format	4 octets (e.g., 192.168.2.33)	16 octets (8 fields) [2]
Field Size	8-bit fields	16-bit fields [2]
Separator	Dot (.)	Colon (:) [2]
Range	0-255 per field	Hexadecimal values [2]
Example	192.168.2.33	FDEC:BA98:7654:3210:ADEC:BDFF:2990:FFFF [2]

Common Misconception: Many students confuse the number of bits with the number of fields. Remember: IPv4 is 32-bit with 4 octets, IPv6 is 128-bit with 8 fields.

Uniform Resource Locator (URL) Structure

URLs provide the complete addressing scheme to locate web resources. A URL consists of four essential parts [2]:

URL Components:

```
http://www.chicagosymphony.org/civicconcerts/index.htm
|____| |____________________| |_____________| |________|
Protocol    Domain Name          Pathname       Filename
```

Component Breakdown:

1. **Protocol:** Specifies the transfer method (e.g., HTTP, HTTPS) [2]
2. **Domain Name:** Identifies the computer hosting the file [2]
3. **Pathname:** Directory location on the server [2]
4. **Filename:** Specific file to access [2]

Domain Name System (DNS)

Purpose: DNS resolves human-readable domain names to IP addresses, enabling users to access websites using memorable names instead of numeric addresses [2].

DNS Parts Breakdown:

```
http://www.tinydancinghorse.com
|____| |_| |_____________| |___|
Protocol Sub   Domain Name  TLD
        domain             (Top-Level Domain)
```

Key Components:

- **Subdomain:** Optional section (e.g., "www", "mail", "blog") representing specific website functions [2]
- **Second-Level Domain (SLD):** Core identifier (e.g., "tinydancinghorse") representing the website or organization [2]
- **Top-Level Domain (TLD):** Highest level (e.g., ".com", ".org", ".edu") categorizing domain purpose or location [2]

HTTP (Hypertext Transfer Protocol)

Definition: HTTP is an application layer protocol governing data transfer between web clients (browsers) and web servers [2].

HTTP Characteristics

Characteristic	Description
Request-Response Model	Client initiates request, server sends response [2]
Stateless Protocol	Each request is independent; no memory of previous requests [2]
Text-Based	Messages are human-readable text [2]

HTTP Methods (Verbs)

Method	Purpose	Example Use Case
GET	Retrieve data from server	Requesting a web page [2]
POST	Send data to create/update resource	Submitting a form [2]
PUT	Update existing resource	Modifying user profile [2]
DELETE	Remove resource	Deleting a post [2]
HEAD	Retrieve headers only	Checking if resource exists [2]
OPTIONS	Retrieve communication options	Discovering available methods [2]

HTTP Status Codes

Code	Meaning	Description
200	OK	Request successful, data returned [2]
404	Not Found	Requested resource doesn't exist [2]
500	Internal Server Error	Server encountered an error [2]

Request and Response Structure:

```
HTTP Request:
GET /up3f/cs4640/syllabus.html HTTP/1.1
```

```
Host: cs.virginia.edu
Accept: text/html

HTTP Response:
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
&lt;html&gt;...&lt;/html&gt;
```

TCP/IP Protocol Suite

Definition: TCP/IP (Transmission Control Protocol/Internet Protocol) is a suite of networking protocols serving as the foundation for internet communication [2].

Key Functions:

- Provides rules and conventions for data exchange between network devices
- Ensures reliable and efficient data transmission
- Fundamental to modern networking infrastructure [2]

Client-Server Architecture

Web Server: A computer program that delivers web pages to users' web browsers [2].

Web Server Functions:

1. **Hosting:** Stores website files (HTML, images, videos, stylesheets, scripts) [2]
2. **Listening for Requests:** Continuously monitors for incoming HTTP/HTTPS requests on specific ports (80 for HTTP, 443 for HTTPS) [2]
3. **Processing Requests:** Examines requested resources and determines appropriate handler [2]
4. **Generating Responses:** Creates HTTP responses, potentially executing server-side scripts (PHP, Python, Node.js) [2]
5. **Content Delivery:** Sends responses back to clients via HTTP/HTTPS [2]

Popular Web Server Software:

- Apache HTTP Server
- Nginx
- Microsoft IIS (Internet Information Services)
- LiteSpeed [2]

Web Client (Browser): Software application enabling users to access and interact with web content [2].

Web Client Functions:

- **Requesting Web Pages:** Sends HTTP requests to web servers [2]
- **Rendering Content:** Parses and displays HTML, CSS, JavaScript [2]
- **Supporting User Interactions:** Handles navigation, forms, links, data submission [2]
- **Caching:** Stores assets locally to optimize performance [2]

Popular Web Browsers:

- Google Chrome
- Mozilla Firefox

- Microsoft Edge
- Apple Safari [2]

Unit 1.2: HTML Fundamentals

Introduction to HTML

HTML (HyperText Markup Language): The standard markup language for creating web pages [2].

Key Characteristics:

- Describes the structure of web pages
- Consists of a series of elements
- Elements tell the browser how to display content
- Labels pieces of content (headings, paragraphs, links, etc.) [2]

HTML Document Structure

```
&lt;html&gt;
&lt;head&gt;
    &lt;title&gt;Why I Love This Course&lt;/title&gt;
&lt;/head&gt;
&lt;body&gt;

&lt;/body&gt;
&lt;/html&gt;
```

Document Components:

1. ``**:** Declares document type as HTML5 [2]
2. `<html>`**:** Root element containing all content [2]
3. `<head>`**:** Contains metadata and document information [2]
4. `<title>`**:** Specifies page title shown in browser tab [2]
5. `<body>`**:** Contains visible page content [2]

HTML Element Anatomy

```
<h1>Hello World!</h1>
|_| |___________| |__|
 |       |         |
Opening Content  Closing
 Tag              Tag
```

Element Components:

- **Opening Tag:** Marks element start
- **Content:** Information displayed
- **Closing Tag:** Marks element end [2]

Self-Closing Tags: Some elements don't require closing tags:

- `<br>` (line break)
- `<hr>` (horizontal rule)
- `<img>` (image) [2]

HTML Semantic Elements

Definition: Semantic elements clearly describe their meaning to both browsers and developers [2].

Structural Semantic Elements

Element	Purpose	Example Use
`<header>`	Introductory content or navigation	Site header with logo and nav [2]
`<nav>`	Navigation links section	Main menu, footer links [2]
`<main>`	Primary document content	Main article or page content [2]
`<article>`	Independent, self-contained content	Blog post, news article [2]
`<section>`	Thematic grouping of content	Chapter, topic section [2]
`<aside>`	Tangentially related content	Sidebar, related links [2]
`<footer>`	Footer for document or section	Copyright info, contact details [2]
`<figure>`	Self-contained content like images	Image with caption [2]

Best Practice: Use semantic elements instead of generic `<div>` elements where appropriate to improve accessibility and SEO [3] [4].

Example - Semantic Structure:

```
&lt;article&gt;
    &lt;header&gt;
        <h1>Understanding Web Development</h1>
    &lt;/header&gt;
    &lt;section&gt;
        <h2>Introduction</h2>
        <p>Web development encompasses...</p>
    &lt;/section&gt;
    &lt;footer&gt;
        <p>Author: John Doe</p>
    &lt;/footer&gt;
&lt;/article&gt;
```

Text Semantic Elements

Element	Meaning	Default Display	When to Use
`<h1>` **to** `</h1><h6>`	Heading levels	Decreasing font size	Content hierarchy [2] [4]
`<p>`	Paragraph	Block with margins	Separating text blocks [4]
`<strong>`	Strong importance	Bold	Critical information [4]
`<em>`	Emphasis	Italic	Stressed text [4]

Element	Meaning	Default Display	When to Use
`<a>`	Hyperlink	Blue underlined	Navigation links [4]
`</a></em></strong></p><ol><strong><em><a>`	Ordered list	Numbered list	Sequential items [4]
`</a><ul><a>`	Unordered list	Bulleted list	Non-sequential items [4]
`&lt;q&gt;`	Short quotation	Inline quote marks	Brief quotes [4]
`</a><blockquote><a>`	Long quotation	Indented block	Extended quotes [4]
`<code>`	Code snippet	Monospace font	Programming code [4]

Common Mistake: Using `<strong>` just for bold styling or `<em>` just for italics. Use CSS for styling; use semantic tags to convey meaning [4].

Block-Level vs Inline Elements

Block-Level Elements

Characteristics:

- Render on new line by default
- May contain inline or other block-level elements
- Take up full width available [2]

Examples:

- `</em></strong></code></a><code><strong><em><div><a>`, `<p>`, `</p><h1>-</h1></a><h6><a>`, `&lt;header&gt;`, `&lt;footer&gt;`, `&lt;section&gt;`, `&lt;article&gt;`, `&lt;nav&gt;`, `&lt;table&gt;`, `&lt;form&gt;`, `</a><ul><a>`, `</a><ol><a>` [2]

Inline Elements

Characteristics:

- Render on same line by default
- May only contain other inline elements
- Take up only necessary width [2]

Examples:

- `<span>`, `</span></a><a>`, `<img>`, `<strong>`, `<em>`, `<code>`, `&lt;button&gt;`, `&lt;input&gt;`, `&lt;label&gt;` [2]

HTML Lists

Unordered Lists

```
<ul>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ul>
```

Output:

- HTML
- CSS
- JavaScript [2]

Ordered Lists

```
<ol>
    <li>Plan structure</li>
    <li>Write HTML</li>
    <li>Apply CSS</li>
    <li>Add JavaScript</li>
</ol>
```

Output:

1. Plan structure
2. Write HTML
3. Apply CSS
4. Add JavaScript [2]

HTML Links

Basic Link Syntax:

```
</code></em></strong></a><strong><em><code><a href="URL">Link Text</a>
```

Link Types:

Type	Example	Purpose
External	`<a href="https://example.com">Visit</a>`	Link to external site [2]
Internal	`<a href="about.html">About</a>`	Link to same-site page [2]
Anchor	`<a href="#top">Go to Top</a>`	Link to page section [2]
Email	`<a href="mailto:user@example.com">Email</a>`	Open email client [2]
Phone	`<a>Call</a>`	Dial phone number [2]

Link Attributes:

- `href`: Specifies destination URL [2]
- `target`: Specifies where to open link (`_blank` for new tab)
- `title`: Provides tooltip text

HTML Images

Basic Image Syntax:

```
<img>
```

Image Attributes:

Attribute	Purpose	Required?
`src`	Image source path/URL	Yes [2]
`alt`	Alternative text for accessibility	Yes [2]
`width`	Image width in pixels	Recommended [2]
`height`	Image height in pixels	Recommended [2]

Why Specify Width and Height: Prevents layout shift as page loads, improving user experience [2].

Unit 1.3: HTML Tables

Table Structure

Basic Table Elements:

```
&lt;table&gt;
    &lt;tr&gt;
        &lt;th&gt;Header 1&lt;/th&gt;
        &lt;th&gt;Header 2&lt;/th&gt;
    &lt;/tr&gt;
    &lt;tr&gt;
        &lt;td&gt;Data 1&lt;/td&gt;
        &lt;td&gt;Data 2&lt;/td&gt;
    &lt;/tr&gt;
&lt;/table&gt;
```

Table Elements:

Element	Purpose
`<table>`	Defines the table container [2]
`<tr>`	Table row [2]
`<th>`	Table header cell (bold, centered by default) [2]
`<td>`	Table data cell [2]
`<thead>`	Groups header content
`<tbody>`	Groups body content
`<tfoot>`	Groups footer content

Advanced Table Attributes:

```
&lt;table&gt;
    &lt;tr&gt;
        &lt;th colspan="3"&gt;Sample Table&lt;/th&gt;
    &lt;/tr&gt;
    &lt;tr&gt;
        &lt;td rowspan="2"&gt;Merged Cell&lt;/td&gt;
        &lt;td&gt;data2&lt;/td&gt;
        &lt;td&gt;data3&lt;/td&gt;
    &lt;/tr&gt;
&lt;/table&gt;
```

- `colspan`: Merges cells horizontally [2]
- `rowspan`: Merges cells vertically [2]

Unit 1.4: HTML Forms

Form Fundamentals

Form Element:

```
&lt;form method="POST" action="process.php"&gt;

&lt;/form&gt;
```

Form Attributes:

Attribute	Values	Purpose
`method`	GET, POST	HTTP method for submission [2]
`action`	URL	Server endpoint for processing [2]
`autocomplete`	on, off	Enable/disable autocomplete [2]
`novalidate`	boolean	Disable browser validation [2]

GET vs POST Methods

Aspect	GET	POST
Visibility	Data visible in URL	Data hidden in request body [2]
Security	Not secure (visible)	More secure [2]
Data Length	Limited (~3000 chars)	Unlimited [2]
Use Case	Non-sensitive data, bookmarkable	Sensitive data, large data [2]
Caching	Can be cached	Not cached [2]

Input Types

Text Input Types

```
&lt;input type="text" name="username" size="25" maxlength="50"&gt;

&lt;input type="password" name="password"&gt;

&lt;input type="email" name="email"&gt;

&lt;input type="url" name="website"&gt;

&lt;input type="tel" name="phone"&gt;
```

```
&lt;input type="search" name="query"&gt;
```

Number and Range Inputs

```
&lt;input type="number" name="quantity" min="1" max="100"&gt;

&lt;input type="range" name="volume" min="0" max="100"&gt;
```

Date and Time Inputs

```
&lt;input type="date" name="birthday" min="2010-12-17" max="2017-11-22"&gt;

&lt;input type="time" name="appointment"&gt;

&lt;input type="datetime-local" name="meeting"&gt;

&lt;input type="month" name="bdaymonth"&gt;

&lt;input type="week" name="week_year"&gt;
```

Selection Inputs

```
&lt;input type="radio" name="gender" value="male"&gt; Male<br>
&lt;input type="radio" name="gender" value="female" checked&gt; Female<br>

&lt;input type="checkbox" name="choice" value="cb1" checked&gt; Sandwich<br>
&lt;input type="checkbox" name="choice" value="cb2"&gt; Pancake<br>

&lt;input type="color" name="favcolor" value="#ff0022"&gt;
```

Button Inputs

```
&lt;input type="submit" value="Submit"&gt;

&lt;input type="reset" value="Reset"&gt;

&lt;button type="button" onclick="alert('Clicked!')"&gt;Click Me!&lt;/button&gt;
```

Input Attributes:

Attribute	Purpose	Example
`name`	Identifies field in submission	`name="username"` [2]
`value`	Default or current value	`value="John"` [2]
`placeholder`	Hint text	`placeholder="Enter name"`
`required`	Makes field mandatory	`required`
`readonly`	Makes field read-only	`readonly` [2]
`disabled`	Disables field	`disabled` [2]
`maxlength`	Maximum character length	`maxlength="50"` [2]
`min`/`max`	Value range limits	`min="1" max="100"` [2]

Textarea Element

```
&lt;textarea name="message" rows="5" cols="30"&gt;
Default text here
&lt;/textarea&gt;
```

Attributes:

- `rows`: Visible lines [2]
- `cols`: Visible width [2]

Select Dropdown

```
&lt;select name="cars"&gt;
    &lt;option value="bmw" selected&gt;BMW&lt;/option&gt;
    &lt;option value="mercedes"&gt;Mercedes&lt;/option&gt;
    &lt;option value="audi"&gt;Audi&lt;/option&gt;
&lt;/select&gt;


&lt;select name="colors" size="3" multiple&gt;
    &lt;option value="red"&gt;Red&lt;/option&gt;
    &lt;option value="green"&gt;Green&lt;/option&gt;
    &lt;option value="blue"&gt;Blue&lt;/option&gt;
&lt;/select&gt;
```

Attributes:

- `size`: Number of visible options [2]
- `multiple`: Allow multiple selections [2]

Form Labels

```


&lt;label for="username"&gt;Username:&lt;/label&gt;
&lt;input type="text" id="username" name="username"&gt;


&lt;label&gt;
```

```
    Email:
    &lt;input type="email" name="email"&gt;
&lt;/label&gt;
```

Benefits of Labels:

- Improves accessibility [2]
- Increases clickable area
- Helps screen readers

Fieldset and Legend

```
&lt;fieldset&gt;
    &lt;legend&gt;Personal Information&lt;/legend&gt;
    Name: &lt;input type="text" name="name"&gt;<br>
    Age: &lt;input type="number" name="age"&gt;
&lt;/fieldset&gt;
```

Purpose:

- `<fieldset>`: Groups related form elements [2]
- `<legend>`: Provides caption for fieldset [2]

Complete Form Example

```
&lt;form action="process.php" method="POST"&gt;
    &lt;fieldset&gt;
        &lt;legend&gt;Contact Form&lt;/legend&gt;

        &lt;label for="name"&gt;Name:&lt;/label&gt;
        &lt;input type="text" id="name" name="name" required&gt;<br>

        &lt;label for="email"&gt;Email:&lt;/label&gt;
        &lt;input type="email" id="email" name="email" required&gt;<br>

        &lt;label for="message"&gt;Message:&lt;/label&gt;
        &lt;textarea id="message" name="message" rows="5" cols="30"&gt;&lt;/textarea&gt;<br>

        &lt;input type="submit" value="Send"&gt;
        &lt;input type="reset" value="Clear"&gt;
    &lt;/fieldset&gt;
&lt;/form&gt;
```

Module 2: Cascading Style Sheets (CSS)

Unit 2.1: CSS Fundamentals

Introduction to CSS

CSS (Cascading Style Sheets): A language for controlling the look and feel of HTML documents in an organized and efficient manner [5].

Purpose:

- Defines HOW HTML elements are displayed
- Separates content (HTML) from presentation (CSS)
- Enables consistent styling across multiple pages [5]

Established by: World Wide Web Consortium (W3C) [5]

CSS Syntax

CSS Rule Structure:

```
selector {
    property: value;
    property: value;
}
```

Example:

```
h1 {
    color: blue;
    font-size: 12px;
}
```

Components:

- **Selector:** HTML element to style [5]
- **Declaration:** Property-value pair [5]
- **Property:** Style attribute to change [5]
- **Value:** Setting for the property [5]
- **Semicolon:** Ends each declaration [5]
- **Curly Brackets:** Surround declaration group [5]

Types of CSS

1. Inline CSS

Definition: CSS properties applied directly within HTML tags using the `style` attribute [5].

```
<p>
    This is a paragraph.
</p>
```

Advantages:

- Quick styling for single elements
- High specificity (overrides other styles) [5]

Disadvantages:

- Not reusable
- Difficult to maintain
- Mixes content with presentation

When to Use: Temporary or unique styles for specific elements [5]

2. Internal CSS

Definition: CSS rules defined within `<style>` tag in the `<head>` section [5].

```
&lt;head&gt;
    &lt;style type="text/css"&gt;
        hr { color: red; }
        p { margin-left: 20px; }
        body { background-image: url("images/back40.gif"); }
    &lt;/style&gt;
&lt;/head&gt;
```

Advantages:

- Single document styling
- No external file needed
- Useful for unique page styles [5]

Disadvantages:

- Not reusable across pages
- Increases HTML file size

When to Use: Single-page unique styling [5]

3. External CSS

Definition: CSS rules stored in separate `.css` file, linked via `<link>` tag [5].

```
&lt;head&gt;
    &lt;link rel="stylesheet" type="text/css" href="mystyle.css"&gt;
&lt;/head&gt;
```

mystyle.css:

```
hr { color: sienna; }
p { margin-left: 20px; }
body { background-image: url("images/back40.gif"); }
```

Advantages:

- Reusable across multiple pages
- Easier maintenance
- Smaller HTML files
- Can change entire site style by modifying one file [5]

Disadvantages:

- Requires additional HTTP request
- May not load if external file unavailable

When to Use: Styling multiple pages, professional websites [5]

Best Practice: External CSS is the recommended approach for most web development projects [5].

Cascading Order

How CSS determines which rules apply when multiple styles target the same element:

Priority Order (Lowest to Highest):

1. Browser default styles
2. External style sheet
3. Internal style sheet (in `<head>`)
4. Inline style (inside HTML element) [5]

Key Principle: Inline styles have the highest priority and will override all other styles [5].

Important Note: If an external stylesheet link is placed *after* an internal stylesheet in the HTML `<head>`, the external stylesheet will override the internal one [5].

CSS Cascade Resolution

When declarations conflict, the cascade considers three factors:

1. **Stylesheet Origin:** Where styles come from (browser default, external, internal, inline) [5]
2. **Selector Specificity:** Which selectors take precedence [5]
3. **Source Order:** Order of style declaration [5]

Decision Flowchart:

```
Conflicting Declarations
  ↓
Different Origin/Importance? → Yes → Use higher-priority origin
  ↓ No
Inline Style? → Yes → Use inline declaration
  ↓ No
Different Specificity? → Yes → Use higher specificity
  ↓ No
Use later declaration (source order)
```

CSS Specificity

Definition: Specificity determines which CSS rule applies when multiple rules target the same element [5].

Specificity Hierarchy

Specificity Order (Lowest to Highest):

1. **Element Selector:** Lowest specificity
2. **Class Selector:** Medium specificity
3. **ID Selector:** Highest specificity
4. **Inline Styles:** Override all specificity [5]

Example - Source Order:

```
h1 { color: red; }
h1 { color: purple; }  /* This wins (later in source) */
```

Output: Purple heading [5]

Example - Specificity:

```
.main-heading { color: red; }  /* This wins (higher specificity) */
h1 { color: blue; }
```

```
<h1>This is my heading.</h1>
```

Output: Red heading [5]

Specificity Rules:

Selector Type	Specificity Level	Wins Over
Element Selector	Low	None (base level) [5]
Class Selector	Medium	Element selectors [5]
ID Selector	High	Class and element selectors [5]
Inline Style	Highest	All other selectors [5]

Resolution Rules:

- If two rulesets have **same specificity**, later one wins
- If they have **different specificity**, higher specificity wins
- **Inline styles** win over any specificity [5]

Unit 2.2: CSS Selectors

Simple Selectors

1. Element Selector

Syntax: Element name

```
p {
    color: blue;
}
```

Targets: All `<p>` elements [5]

Output:

```
</p><p>Blue text</p>
<p>Blue text</p>
```

2. Class Selector

Syntax: Dot (`.`) followed by class name

```
.blue {
    color: blue;
}
```

Targets: All elements with `class="blue"` [5]

Output:

```
<p>Blue text</p>
<div>Blue text</div>
<p>Unaffected</p>
```

3. ID Selector

Syntax: Hash (`#`) followed by ID name

```
#name {
    color: blue;
}
```

Targets: Element with `id="name"` [5]

Output:

```
<div>Blue text</div>
<p>Unaffected</p>
```

Important: IDs must be unique on a page; use classes for multiple elements [5]

4. Universal Selector

Syntax: Asterisk (`*`)

```
* {
    margin: 0;
    padding: 0;
}
```

Targets: All elements on the page [5]

Selector Comparison Table

Aspect	Element Selector	Class Selector	ID Selector
Syntax	p	`.my-class`	`#my-id` [5]
Specificity	Low	Medium	High [5]
Reusability	All elements of type	Multiple elements	Single element [5]

Aspect	Element Selector	Class Selector	ID Selector
HTML Example	`<p>Text</p>`	`<a>Link</a>`	`<div>Unique</div>` [5]
Use Case	Styling all paragraphs	Styling multiple similar elements	Styling unique element [5]

Grouping Selectors

Multiple Styles for One Selector

```
h2 {
    color: darkblue;
    font-style: italic;
}
```

Multiple Selectors with Same Styles

```
h1, h2, h3 {
    color: yellow;
}
```

Separator: Comma (`,`) [5]

Combining Selectors

Element with Class Selector

Syntax: Element name immediately followed by class name (no space)

```
p.big {
    font-size: 20px;
}
```

Targets: Every `<p>` element that has `class="big"` [5]

Output:

```
</p><p>Text size 20px</p>
<div>Unaffected</div>
```

Note the lack of space between element and class – this is crucial [5]

Combinator Selectors

Definition: Combinators explain relationships between selectors [5]

1. Descendant Selector (Space)

Syntax: `ancestor descendant`

```
article p {
    color: blue;
```

```
}
```

Targets: Every `<p>` inside `<article>` at any level [5]

Example:

```
&lt;article&gt;
    </p><p>Blue text</p>
    <div>
        <p>Blue text</p>
    </div>
&lt;/article&gt;
<p>Unaffected</p>
```

2. Child Selector (`>`)

Syntax: `parent > child`

```
article &gt; p {
    color: blue;
}
```

Targets: Every `<p>` that is a **direct child** of `<article>` [5]

Example:

```
&lt;article&gt;
    </p><p>Blue text</p>
    <div>
        <p>Unaffected</p>
    </div>
&lt;/article&gt;
```

3. Adjacent Sibling Selector (`+`)

Syntax: `element1 + element2`

Targets: `element2` immediately following `element1` [5]

4. General Sibling Selector (`~`)

Syntax: `element1 ~ element2`

Targets: All `element2` siblings after `element1` [5]

Pseudo-Classes

Definition: Used to define special states of elements [5]

Syntax:

```
selector:pseudo-class {
    property: value;
}
```

Common Pseudo-Classes:

Pseudo-Class	Purpose	Example
`:hover`	Element when mouse hovers	`a:hover { color: red; }` [5]
`:active`	Element when clicked	`button:active { background: gray; }` [5]
`:focus`	Element when focused	`input:focus { border: blue; }` [5]
`:visited`	Visited links	`a:visited { color: purple; }`
`:nth-child(n)`	nth child element	`li:nth-child(2) { ... }`

Link States:

```
a:link { color: blue; }        /* Unvisited link */
a:visited { color: purple; }   /* Visited link */
a:hover { color: red; }        /* Mouse over */
a:active { color: yellow; }    /* Being clicked */
```

Pseudo-Elements

Definition: Used to style specific parts of elements [5]

Syntax:

```
selector::pseudo-element {
    property: value;
}
```

Common Pseudo-Elements:

Pseudo-Element	Purpose	Example
`::first-line`	First line of text	`p::first-line { font-weight: bold; }` [5]
`::first-letter`	First letter	`p::first-letter { font-size: 2em; }` [5]
`::before`	Insert content before	`h1::before { content: "→ "; }` [5]
`::after`	Insert content after	`h1::after { content: " ←"; }` [5]

Unit 2.3: CSS Properties

Color and Text Properties

Color Property

Sets text color:

```
body { color: blue; }
h1 { color: #00ff00; }
h2 { color: rgb(255, 0, 0); }
```

Color Formats:

- Named colors: `red`, `blue`, `green`
- Hex: `#ff0000`
- RGB: `rgb(255, 0, 0)`
- RGBA: `rgba(255, 0, 0, 0.5)` (with opacity) [5]

Font Properties

Property	Purpose	Example
`font-family`	Specifies font	`font-family: Arial, sans-serif;` [5]
`font-size`	Sets font size	`font-size: 14px;` [5]
`font-weight`	Controls thickness	`font-weight: bold;` [5]
`font-style`	Normal/italic/oblique	`font-style: italic;` [5]
`font-variant`	Small caps	`font-variant: small-caps;` [5]

Font Family Rules:

- Multi-word names require quotes: `font-family: "Times New Roman";`
- Provide fallbacks: `font-family: Arial, Helvetica, sans-serif;` [5]

Font Weight Values:

- Keywords: `normal`, `bold`, `bolder`, `lighter`
- Numeric: `100` (thin) to `900` (thick), in increments of 100 [5]

Font Shorthand:

```
p {
    font: font-style font-variant font-weight font-size/line-height font-family;
}

/* Example */
p {
    font: italic small-caps bold 14px/1.5 Arial, sans-serif;
}
```

Order must be: style, variant, weight, size/line-height, family [5]

Text Properties

Property	Purpose	Values
`text-align`	Horizontal alignment	`left`, `right`, `center`, `justify` [5]
`text-decoration`	Text decorations	`underline`, `overline`, `line-through`, `none` [5]
`text-transform`	Case transformation	`uppercase`, `lowercase`, `capitalize` [5]
`text-indent`	First line indentation	`50px`, `2em` [5]
`letter-spacing`	Space between letters	`3px` [5]

Property	Purpose	Values
`word-spacing`	Space between words	`10px` [5]
`line-height`	Space between lines	`1.8`, `20px` [5]
`direction`	Text direction	`ltr`, `rtl` [5]

Examples:

```
h1 { text-align: center; }
h2 { text-decoration: underline; }
p { text-transform: uppercase; }
p { text-indent: 50px; }
p { letter-spacing: 3px; }
p { word-spacing: 10px; }
p { line-height: 1.8; }
p { direction: rtl; }
```

Text Decoration for Links:

```
a { text-decoration: none; }  /* Remove underline */
```

Background Properties

Property	Purpose	Example
`background-color`	Sets background color	`background-color: lightblue;` [5]
`background-image`	Sets background image	`background-image: url("image.jpg");` [5]
`background-repeat`	Controls image repeat	`background-repeat: no-repeat;` [5]
`background-size`	Sets image size	`background-size: cover;` [5]
`background-position`	Sets image position	`background-position: center;`
`background-attachment`	Scroll behavior	`background-attachment: fixed;`

Background Repeat Values:

- `repeat` (default)
- `repeat-x` (horizontal only)
- `repeat-y` (vertical only)
- `no-repeat` [5]

Background Shorthand:

```
body {
    background: #ffffff url("img_tree.png") no-repeat right top;
}
```

Order: color, image, repeat, attachment, position [5]

Border Properties

Border Style Values:

Value	Description
`dotted`	Dotted border [5]
`dashed`	Dashed border [5]
`solid`	Solid border [5]
`double`	Double border [5]
`groove`	3D grooved border [5]
`ridge`	3D ridged border [5]
`inset`	3D inset border [5]
`outset`	3D outset border [5]
`none`	No border [5]
`hidden`	Hidden border [5]

Border Properties:

```
p {
    border-width: 5px;
    border-style: solid;
    border-color: red;
}
```

Border Shorthand:

```
p {
    border: 5px solid red;
}
```

Order: width, style (required), color [5]

Individual Sides:

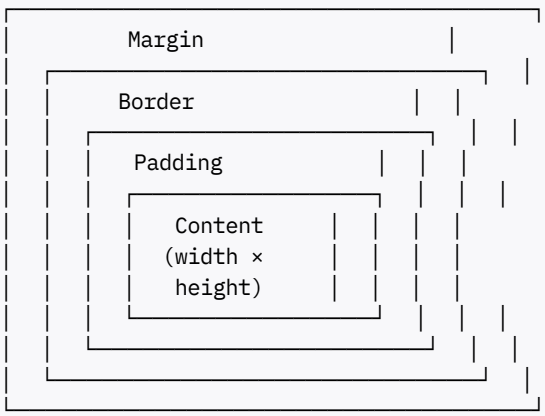
```
p {
    border-top: 2px solid blue;
    border-right: 3px dashed green;
    border-bottom: 4px dotted red;
    border-left: 5px double black;
}
```

Unit 2.4: CSS Box Model

Box Model Concept

Definition: The CSS box model is essentially a box that wraps around every HTML element, consisting of margins, borders, padding, and content [5] [6].

Box Model Layers (Inside to Outside):



Box Model Components

Component	Description	Transparency
Content	Where text and images appear	Opaque [5] [6]
Padding	Space around content	Transparent [5] [6]
Border	Frame around padding and content	Can be styled [5] [6]
Margin	Space outside border	Transparent [5] [6]

Calculating Element Dimensions

Total Width Calculation:

```
Total Width = width + left padding + right padding + left border + right border + left margin + righ
```

Example:

```
div {
    width: 320px;
    height: 50px;
    padding: 10px;
    border: 5px solid gray;
    margin: 0;
}
```

Calculations:

- **Content width:** 320px
- **Left padding + right padding:** 10px + 10px = 20px
- **Left border + right border:** 5px + 5px = 10px
- **Total width:** 320px + 20px + 10px = **350px** [7]

Total Height:

- **Content height:** 50px
- **Top padding + bottom padding:** 10px + 10px = 20px
- **Top border + bottom border:** 5px + 5px = 10px
- **Total height:** 50px + 20px + 10px = **80px** [7]

Padding Property

Purpose: Creates space between content and border [5] [6]

Syntax:

```
/* All sides */
div { padding: 10px; }

/* Vertical | Horizontal */
div { padding: 10px 20px; }

/* Top | Right | Bottom | Left (clockwise) */
div { padding: 10px 20px 15px 25px; }

/* Individual sides */
div {
    padding-top: 10px;
    padding-right: 20px;
    padding-bottom: 15px;
    padding-left: 25px;
}
```

Border Property

Individual Properties:

```
div {
    border-width: 5px;
    border-style: solid;
    border-color: gray;
}
```

Shorthand:

```
div {
    border: 5px solid gray;
}
```

Individual Sides:

```
div {
    border-top: 2px solid red;
    border-right: 3px dashed blue;
    border-bottom: 4px dotted green;
    border-left: 5px double black;
}
```

Margin Property

Purpose: Creates space outside border [5] [6]

Syntax:

```
/* All sides */
div { margin: 10px; }

/* Vertical | Horizontal */
div { margin: 10px 20px; }

/* Top | Right | Bottom | Left */
div { margin: 10px 20px 15px 25px; }

/* Individual sides */
div {
    margin-top: 10px;
    margin-right: 20px;
    margin-bottom: 15px;
    margin-left: 25px;
}
```

Auto Centering:

```
div {
    width: 500px;
    margin: 0 auto;  /* Centers horizontally */
}
```

Margin Collapse

Definition: When vertical margins of adjacent elements touch, they collapse into a single margin [6].

Behavior:

- **Horizontal margins:** Do NOT collapse (cumulative)
- **Vertical margins:** DO collapse (larger margin wins) [6]

Example:

```
.box1 { margin-bottom: 20px; }
.box2 { margin-top: 30px; }
```

Result: Total space between boxes is **30px** (not 50px) [6]

Horizontal Example:

```
.box1 { margin-right: 40px; }
.box2 { margin-left: 50px; }
```

Result: Total space is **90px** (cumulative) [6]

Preventing Margin Collapse:

- Use padding instead of margins

- Add border or padding to parent
- Use flexbox or grid layout

Module 3: Advanced CSS - Layout and Styling

Unit 3.1: Styling Specific Elements

Styling Links

Link Pseudo-Class States:

```
a:link { color: blue; }        /* Unvisited link */
a:visited { color: purple; }   /* Visited link */
a:hover { color: red; }        /* Mouse over */
a:active { color: yellow; }    /* Being clicked */
```

Order Matters: The order should be `:link`, `:visited`, `:hover`, `:active` (remember: **LoVe HAte**) [6]

Styling Tables

Table Properties:

Property	Purpose	Example
`border`	Table borders	`border: 1px solid black;` [6]
`border-collapse`	Border spacing	`border-collapse: collapse;` [6]
`border-spacing`	Space between borders	`border-spacing: 10px;` [6]
`width` / `height`	Table dimensions	`width: 100%;` [6]
`text-align`	Horizontal alignment	`text-align: center;` [6]
`vertical-align`	Vertical alignment	`vertical-align: top;` [6]
`background-color`	Background color	`background-color: lightgray;` [6]
`padding`	Cell padding	`padding: 10px;` [6]

Table Layout:

```
table {
    table-layout: auto;   /* Adjusts based on content */
    table-layout: fixed;  /* Fixed column widths */
}
```

Hoverable Table:

```
tr:hover {
    background-color: gray;
}
```

Example - Complete Table Styling:

```
table {
    border: 1px solid black;
    border-collapse: collapse;
    width: 100%;
}

th, td {
    border: 1px solid black;
    padding: 10px;
    text-align: left;
}

th {
    background-color: blueviolet;
    color: white;
}

tr {
    background-color: hotpink;
}

tr:hover {
    background-color: blue;
}
```

Styling Forms

Input Field Properties:

Property	Purpose	Example
`width`	Input width	`width: 100%;` [6]
`padding`	Space inside input	`padding: 10px;` [6]
`margin`	Space outside input	`margin: 10px 0;` [6]
`border`	Input border	`border: 2px solid #ccc;` [6]
`border-radius`	Rounded corners	`border-radius: 4px;` [6]
`box-shadow`	Shadow effect	`box-shadow: 0 0 5px rgba(0,0,0,0.1);` [6]
`font`	Font properties	`font: 16px Arial;` [6]
`background-color`	Background color	`background-color: #f8f8f8;` [6]

Focused Input:

```
input:focus {
    border: 2px solid blue;
    outline: none;
    background-color: lightyellow;
}
```

Example - Form Styling:

```
input[type="text"],
input[type="email"],
```

```
textarea {
    width: 100%;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box;
    margin-top: 6px;
    margin-bottom: 16px;
}

input[type="text"]:focus,
input[type="email"]:focus,
textarea:focus {
    border: 2px solid #4CAF50;
    outline: none;
}
```

Styling Images

Image Styling Techniques:

Technique	CSS	Purpose
Resize	`width: 500px; height: auto;`	Change size [6]
Rounded	`border-radius: 50%;`	Circular image [6]
Thumbnail	`border: 1px solid #ddd; padding: 5px;`	Frame effect [6]
Responsive	`max-width: 100%; height: auto;`	Adapt to screen [6]
Center	`display: block; margin: 0 auto;`	Horizontal center [6]
Transparent	`opacity: 0.5;`	Semi-transparent [6]

Examples:

```
/* Responsive Image */
img {
    max-width: 100%;
    height: auto;
}

/* Rounded/Circular Image */
img.rounded {
    border-radius: 8px;    /* Rounded corners */
}

img.circle {
    border-radius: 50%;    /* Perfect circle */
}

/* Thumbnail */
img.thumbnail {
    border: 1px solid #ddd;
    border-radius: 4px;
    padding: 5px;
}

/* Centered Image */
img.center {
    display: block;
```

```
    margin-left: auto;
    margin-right: auto;
}

/* Transparent Image */
img.transparent {
    opacity: 0.5;
}

img.transparent:hover {
    opacity: 1;
}
```

Unit 3.2: CSS Layout Techniques

CSS Position Property

Position Values:

Value	Behavior	Use Case
`static`	Default positioning	Normal flow [6]
`relative`	Relative to normal position	Minor adjustments [6]
`fixed`	Fixed to viewport	Sticky headers [6]
`absolute`	Relative to nearest positioned ancestor	Overlays, modals [6]
`sticky`	Switches between relative and fixed	Sticky navigation [6]

Static Positioning

Default behavior: Elements are positioned according to normal document flow [6]

```
p {
    position: static;
}
```

Characteristics:

- Not affected by `top`, `bottom`, `left`, `right` properties
- Default for all elements [6]

Relative Positioning

Positioned relative to its normal position [6]

```
p {
    position: relative;
    top: 50px;
    left: 50px;
}
```

Characteristics:

- Normal document flow position is reserved
- Element moves from its normal position
- Other elements are not affected [6]

Visual:

```
Normal position → Element with relative positioning
                  ↓ (top: 50px, left: 50px)
                  Moved element (space reserved)
```

Absolute Positioning

Positioned relative to nearest positioned ancestor [6]

```
div {
    position: absolute;
    top: 20px;
    left: 30px;
}
```

Characteristics:

- Removed from normal document flow
- Positioned relative to nearest positioned parent
- If no positioned ancestor, positioned relative to `<html>`
- Other elements fill the gap [6]

Fixed Positioning

Positioned relative to viewport [6]

```
header {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
}
```

Characteristics:

- Stays in same position when scrolling
- Removed from normal document flow
- Common for navigation bars [6]

Sticky Positioning

Toggles between relative and fixed [6]

```
nav {
    position: sticky;
```

```
    top: 0;
}
```

Characteristics:

- Acts as `relative` until scroll threshold
- Becomes `fixed` when threshold reached
- Useful for sticky headers [6]

CSS Float Property

Purpose: Float elements to create flexible layouts [6]

Values:

- `left` - Float to left
- `right` - Float to right
- `none` - No floating (default)
- `inherit` - Inherit from parent

Example:

```
img {
    float: left;
    margin-right: 10px;
}
```

Clear Property:

Controls flow next to floated elements [6]

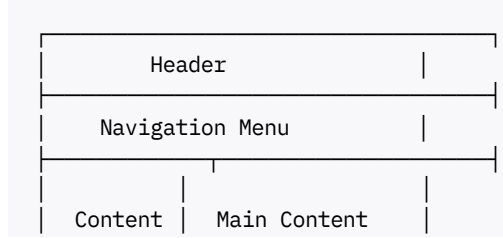
```
.clear {
    clear: both;  /* Prevents wrapping around floated elements */
}
```

Clear Values:

- `left` - No floating elements on left
- `right` - No floating elements on right
- `both` - No floating elements on either side
- `none` - Allows floating elements (default) [6]

CSS Layout Patterns

Common Web Layout Structure:



```
|          |                    |
|----------|--------------------|
|          |                    |
|        Footer                 |
|                               |
```

Layout Components:

Component	Purpose	Typical Location
Header	Branding, logo, top navigation	Top of page [6]
Navigation	Links for site navigation	Below header or sidebar [6]
Content	Main page content	Center [6]
Sidebar	Secondary content	Left or right side [6]
Footer	Copyright, contact info	Bottom of page [6]

Layout Column Configurations:

- **1-column:** Mobile browsers [6]
- **2-column:** Tablets and laptops [6]
- **3-column:** Desktops only [6]

Dropdown Menu

HTML Structure:

```
<div>
    &lt;button class="dropbtn"&gt;Menu&lt;/button&gt;
    <div>
        <a href="#">Link 1</a>
        <a href="#">Link 2</a>
        <a href="#">Link 3</a>
    </div>
</div>
```

CSS:

```
.dropdown {
    position: relative;
    display: inline-block;
}

.dropdown-content {
    display: none;
    position: absolute;
    background-color: #f9f9f9;
    min-width: 160px;
    box-shadow: 0 8px 16px rgba(0,0,0,0.2);
}

.dropdown:hover .dropdown-content {
    display: block;
}
```

Key Concepts:

- Use `position: relative` on container [6]
- Use `position: absolute` on dropdown content [6]
- Toggle visibility with `:hover` pseudo-class [6]

Unit 3.3: CSS Advanced Concepts

CSS Specificity Details

Specificity Calculation:

Different selector types have different weights:

Selector Type	Weight	Example
Inline styles	1000	`style="color: red;"` [6]
IDs	100	`#header` [6]
Classes, attributes, pseudo-classes	10	`.nav`, `[type="text"]`, `:hover` [6]
Elements, pseudo-elements	1	`div`, `::before` [6]

Calculation Example:

```
#header .nav li:hover { }  /* Specificity: 121 (100+10+10+1) */
div p { }                  /* Specificity: 2 (1+1) */
.button { }                /* Specificity: 10 */
```

Higher specificity wins regardless of source order [6]

CSS Inheritance

Inheritance Concept:

```
body element (parent)
  ↓ (inherits styles)
Children elements
  ↓ (inherit from parents)
Grandchildren elements
```

Properties that Inherit:

- Color
- Font properties (family, size, weight, style)
- Text properties (align, indent, transform)
- Line-height
- Visibility [6]

Properties that Don't Inherit:

- Margin
- Padding

- Border
- Background
- Width/Height
- Position [6]

CSS Units

Absolute Units:

Unit	Description	Use Case
`px`	Pixels (fixed size)	Precise control [6]
`pt`	Points (1/72 inch)	Print stylesheets
`cm`, `mm`, `in`	Physical measurements	Print media

Relative Units:

Unit	Relative To	Example
`%`	Parent element size	`width: 50%;` [6]
`em`	Parent element font size	`font-size: 2em;` [6]
`rem`	Root (`<html>`) font size	`padding: 1.5rem;` [6]
`vw`	Viewport width (1vw = 1% of viewport width)	`width: 50vw;`
`vh`	Viewport height (1vh = 1% of viewport height)	`height: 100vh;`

Em vs Rem Example:

```
html { font-size: 16px; }  /* Root size */

.parent {
    font-size: 20px;  /* Parent size */
}

.child-em {
    font-size: 2em;   /* 2 × 20px = 40px (relative to parent) */
}

.child-rem {
    font-size: 2rem;  /* 2 × 16px = 32px (relative to root) */
}
```

Best Practices:

- Use `rem` for consistent sizing throughout site [6]
- Use `em` when size should relate to parent [6]
- Use `%` for responsive widths [6]
- Use `px` when exact size needed [6]

CSS Display Property

Display Values:

Value	Behavior
`block`	Element takes full width, starts on new line
`inline`	Element takes only necessary width, stays in line
`inline-block`	Inline but can have width/height
`none`	Element is hidden and takes no space
`flex`	Enables flexbox layout
`grid`	Enables grid layout

Example:

```
/* Hide element */
.hidden {
    display: none;
}

/* Make inline element block-level */
span {
    display: block;
}

/* Make block element inline */
div {
    display: inline;
}
```

CSS Opacity

Opacity Property:

```
img {
    opacity: 0.5;  /* 0 = fully transparent, 1 = fully opaque */
}

img:hover {
    opacity: 1;    /* Full opacity on hover */
}
```

Range: 0.0 (completely transparent) to 1.0 (completely opaque) [6]

Alternative - RGBA:

```
background-color: rgba(255, 0, 0, 0.5);  /* Red with 50% opacity */
```

Difference:

- `opacity` affects entire element including children
- `rgba` affects only specific property [6]

Module 4: JavaScript Fundamentals

Unit 4.1: JavaScript Introduction

What is JavaScript?

Definition: JavaScript is a high-level, interpreted programming language primarily used for building interactive and dynamic web pages [8].

Key Characteristics:

- **Client-side scripting:** Runs directly in web browser [8]
- **Event-driven:** Responds to user actions [8]
- **Dynamic:** Creates interactive features without page reload [8]
- **Interpreted:** Executed line by line, not compiled [8]

Also Used For:

- Server-side programming (Node.js) [8]
- Mobile app development
- Desktop applications
- Game development

JavaScript Applications

Common Uses:

1. **Client-side validation:** Check form data before submission [8]
2. **Dynamic drop-down menus:** Interactive navigation [8]
3. **Displaying date and time:** Real-time information [8]
4. **Pop-up windows and dialogs:** Alert, confirm, prompt boxes [8]
5. **DOM manipulation:** Dynamically update page content [8]
6. **Event handling:** Respond to clicks, keypresses, etc. [8]

Where to Place JavaScript

Three Placement Options:

1. Between `<body>` Tags

```
&lt;body&gt;
    <h1>My Page</h1>
    &lt;script&gt;
        document.write("Hello, World!");
    &lt;/script&gt;
&lt;/body&gt;
```

2. Between `<head>` Tags

```
&lt;head&gt;
    &lt;script&gt;
        function greet() {
            alert("Hello!");
        }
    &lt;/script&gt;
&lt;/head&gt;
```

3. External .js File (Recommended)

```
&lt;head&gt;
    &lt;script src="script.js"&gt;&lt;/script&gt;
&lt;/head&gt;
```

script.js:

```
console.log("External JavaScript loaded");
```

Best Practice: Use external JavaScript files for better maintainability and reusability [8]

Unit 4.2: JavaScript Basics

Variables

Definition: A variable is a name associated with a piece of data, allowing you to store and manipulate data in programs [8].

Declaration:

```
var message = "hi";
```

Key Points:

- Variable definitions should start with `var`, `let`, or `const` [8]
- No types declared (dynamically typed) [8]
- Same variable can hold different types during execution [8]
- JavaScript is case-sensitive (`message` ≠ `Message`)

Variable Naming Rules:

- Must start with letter, underscore (_), or dollar sign ($)
- Can contain letters, numbers, underscores, dollar signs
- Cannot use reserved keywords
- Use camelCase convention (e.g., `firstName`, `userAge`)

Modern Variable Declaration (ES6+)

Three Ways:

Keyword	Scope	Reassignable	Hoisting	Use Case
`var`	Function	Yes	Yes	Legacy code [9] [10]
`let`	Block	Yes	No	Values that change [9] [10]
`const`	Block	No	No	Constants [9] [10]

Examples:

```
var oldStyle = "function-scoped";  // Old way

let age = 25;      // Can be reassigned
age = 26;          // ✓ Valid

const PI = 3.14;   // Cannot be reassigned
PI = 3.15;         // ✗ Error
```

Best Practice: Use `const` by default, `let` when reassignment needed, avoid `var` [9] [10]

Data Types

JavaScript Data Types:

```
Data Types
├── Primitive
│   ├── Number
│   ├── String
│   ├── Boolean
│   ├── Null
│   ├── Undefined
│   └── Symbol (ES6)
└── Non-Primitive
    └── Object
```

Primitive Data Types

1. Number:

```
var x1 = 34.00;      // Decimal
var x2 = 34;         // Integer
var y = 123e5;       // Scientific notation: 12300000
var z = 123e-5;      // 0.00123
```

2. String:

```
var carName = "Volvo XC60";   // Double quotes
var carName = 'Volvo XC60';   // Single quotes
```

3. Boolean:

```
var isActive = true;
var isComplete = false;
```

4. Undefined:

```
var car;  // Value is undefined, type is undefined
```

5. Null:

```
var person = null;  // Value is null, type is object
```

Note: In JavaScript, `null` is an object (this is actually a bug in JavaScript that cannot be fixed for backwards compatibility) [8]

Typeof Operator

Check Data Type:

```
typeof "John"      // Returns "string"
typeof 3.14        // Returns "number"
typeof true        // Returns "boolean"
typeof false       // Returns "boolean"
typeof x           // Returns "undefined" (if x has no value)
typeof null        // Returns "object" (JavaScript quirk)
```

Undefined vs Null

Aspect	Undefined	Null
Meaning	Variable declared but not assigned	Intentional absence of value [8]
Type	`undefined`	`object` [8]
Assignment	Automatic	Manual

Comparison:

```
typeof undefined     // "undefined"
typeof null          // "object"
null === undefined   // false (different types)
null == undefined    // true (same value)
```

Implicit Type Conversion

JavaScript automatically converts types in expressions:

```
var x = 4;
var y = 11;
var z = "cat";
var q = "17";

var ans = x + y;    // 15 (number + number)
```

```
var ans = z + x;    // "cat4" (string + number = string)
var ans = x + q;    // "417" (number + string = string)
```

Rules:

- Number + Number = Number
- String + Number = String
- String + String = String [8]

Unit 4.3: JavaScript Operators

Arithmetic Operators

Operator	Operation	Example	Result
`+`	Addition	`5 + 3`	8 [8]
`-`	Subtraction	`5 - 3`	2 [8]
`*`	Multiplication	`5 * 3`	15 [8]
`/`	Division	`15 / 3`	5 [8]
`%`	Modulus (remainder)	`17 % 5`	2 [8]
`++`	Increment	`x++`	Increases by 1 [8]
`--`	Decrement	`x--`	Decreases by 1 [8]

Relational Operators

Operator	Meaning	Example	Result
`==`	Equality (with type conversion)	`'50' == 50`	`true` [8]
`!=`	Not equal (with type conversion)	`30 != 50`	`true` [8]
`===`	Strict equality (no type conversion)	`'50' === 50`	`false` [8]
`!==`	Strict not equal	`50 !== 50`	`false` [8]
`>`	Greater than	`50 > 40`	`true` [8]
`>=`	Greater than or equal	`50 >= 50`	`true` [8]
`<`	Less than	`50 < 40`	`false` [8]
`<=`	Less than or equal	`50 <= 50`	`true` [8]

Important Distinction: `==` vs `===`

```
var v1 = ("5" == 5);     // true (type conversion performed)
var v2 = ("5" === 5);    // false (no type conversion)
var v3 = (5 === 5.0);    // true (same type and value)
var v4 = (true == 1);    // true (true converted to 1)
var v5 = (true === 1);   // false (different types)
```

Best Practice: Use `===` and `!==` to avoid unexpected type conversions [8]

Logical Operators

Operator	Name	Description
`&amp;&amp;`	AND	Both expressions must be true [8]
`||`	OR	One expression must be true [8]
`!`	NOT	Inverts boolean value [8]

Examples:

```
var x = 5;
var y = 10;

(x &gt; 0 &amp;amp;&amp;amp; y &gt; 0)   // true (both conditions true)
(x &gt; 0 || y &lt; 0)   // true (at least one true)
!(x &gt; 0)           // false (inverts true to false)
```

Truth Tables:

AND (`&amp;&amp;`):

A	B	A && B
T	T	T
T	F	F
F	T	F
F	F	F

OR (`||`):

A	B	A || B
T	T	T
T	F	T
F	T	T
F	F	F

Unit 4.4: JavaScript Control Structures

Conditional Statements

If Statement

```
var number = 15;

if (number &gt; 10) {
    console.log("Number is greater than 10");
}
```

If-Else Statement

```
var number = 5;

if (number &gt; 10) {
    console.log("Number is greater than 10");
} else {
    console.log("Number is 10 or less");
}
```

If-Else-If Statement

```
var score = 75;

if (score &gt;= 90) {
    console.log("Grade: A");
} else if (score &gt;= 80) {
    console.log("Grade: B");
} else if (score &gt;= 70) {
    console.log("Grade: C");
} else {
    console.log("Grade: F");
}
```

Nested If Statement

```
var number = 4;
number = prompt("Enter a number less than 5 or greater than 20");

if (number &lt; 5) {
    console.log("Number is less than 5!");
} else {
    if (number &gt; 20) {
        console.log("Number is greater than 20!");
    }
}
```

Conditional (Ternary) Operator

Syntax: `condition ? expressionIfTrue : expressionIfFalse`

```
var num1 = 5;
var num2 = 4;

var z = (num1 &gt; num2) ? num1 : num2;  // z = 5
console.log(z);
```

Equivalent If-Else:

```
var z;
if (num1 &gt; num2) {
    z = num1;
} else {
```

```
    z = num2;
}
```

Loops

For Loop

```
for (var i = 0; i &lt; 5; i++) {
    console.log("Iteration: " + i);
}

// Output:
// Iteration: 0
// Iteration: 1
// Iteration: 2
// Iteration: 3
// Iteration: 4
```

While Loop

```
var i = 0;
while (i &lt; 5) {
    console.log("Count: " + i);
    i++;
}
```

Do-While Loop

```
var i = 0;
do {
    console.log("Number: " + i);
    i++;
} while (i &lt; 5);
```

Unit 4.5: JavaScript Functions

Function Basics

Definition: Functions are reusable blocks of code that perform specific tasks [8]

Function Declaration:

```
function functionName(parameter1, parameter2) {
    // Code to execute
    return result;
}
```

Example:

```
function add(a, b) {
    return a + b;
```

```
}

var sum = add(5, 3);  // sum = 8
```

Key Concepts:

- **Parameters:** Variables listed in function definition (a, b) [8]
- **Arguments:** Actual values passed when calling function (5, 3) [8]
- **Return:** Sends value back to caller [8]

Function Invocation

Three Ways to Invoke:

```
// 1. Direct call
add(4, 5);  // Executes function, returns result

// 2. Assign to variable
var result = add(4, 5);  // result = 9

// 3. As expression
console.log(add(4, 5));  // Logs: 9
```

Function Types

Function Declaration

```
function greet(name) {
    return "Hello, " + name;
}
```

Function Expression

```
var greet = function(name) {
    return "Hello, " + name;
};
```

Arrow Functions (ES6)

```
// Traditional function
function add(a, b) {
    return a + b;
}

// Arrow function
const add = (a, b) =&gt; a + b;

// Single parameter (parentheses optional)
const double = num =&gt; num * 2;

// No parameters (parentheses required)
const sayHello = () =&gt; "Hello!";
```

```
// Multiple statements (braces required)
const calculate = (a, b) =&gt; {
    const sum = a + b;
    return sum * 2;
};
```

Arrow Function Benefits:

- Concise syntax [9] [10]
- Lexical `this` binding [10]
- Implicit return for single expressions [10]

Function Scope

Definition: Scope determines where variables are accessible [8]

Global Scope

```
var globalVar = "I'm global";

function test() {
    console.log(globalVar);  // Accessible
}
```

Characteristics:

- Declared outside functions
- Accessible everywhere [8]

Function Scope

```
function test() {
    var localVar = "I'm local";
    console.log(localVar);  // Accessible
}

console.log(localVar);  // Error: not defined
```

Characteristics:

- Declared inside functions
- Only accessible within that function [8]

Block Scope (ES6)

```
if (true) {
    let blockVar = "Block scoped";
    const alsoBlock = "Also block scoped";
}

console.log(blockVar);     // Error: not defined
console.log(alsoBlock);    // Error: not defined
```

Scope Chain:

```
Global Scope
  ↓ (accessible)
Function A Scope
  ↓ (accessible)
Function B Scope (nested in A)
  ↓ (accessible)
Block Scope
```

Inner scopes can access outer scopes, but not vice versa [8]

Closures

Definition: A closure is a function that has access to variables from its parent scope, even after the parent function has returned [11] [12].

Example:

```
function counter() {
    let count = 0;

    return function() {
        count++;
        console.log(count);
    };
}

const incrementCounter = counter();
incrementCounter();  // Output: 1
incrementCounter();  // Output: 2
incrementCounter();  // Output: 3
```

How It Works:

1. `counter()` function creates `count` variable [11]
2. Returns inner function that accesses `count` [11]
3. Even after `counter()` finishes, inner function retains access to `count` [11]
4. This "remembering" is the closure [11]

Use Cases:

- Data privacy / encapsulation [11]
- Function factories [11]
- Event handlers [11]
- Callbacks [11]

Function Factory Example:

```
function greet(greeting) {
    return function(name) {
        return `${greeting}, ${name}!`;
    };
}
```

```
const greetHello = greet('Hello');
const greetHi = greet('Hi');

console.log(greetHello('Alice'));  // "Hello, Alice!"
console.log(greetHi('Bob'));       // "Hi, Bob!"
```

Unit 4.6: Document Object Model (DOM)

DOM Fundamentals

Definition: The Document Object Model (DOM) represents the structure of an HTML document as a tree of objects that JavaScript can manipulate [8].

DOM Tree Structure:

```
Document
  └── html
      ├── head
      │   └── title
      │       └── "My Title" (text)
      └── body
          ├── h1
          │   └── "A heading" (text)
          └── a (href="...")
              └── "Link text" (text)
```

JavaScript DOM Capabilities:

- Access and change all HTML elements [8]
- Change all HTML attributes [8]
- Change all CSS styles [8]
- React to all HTML DOM events [8]
- Add and delete HTML elements [8]

DOM Selection Methods

getElementById

Purpose: Selects single element by ID [8] [13]

```
const element = document.getElementById("myId");
```

Example:

```
<h1>Welcome</h1>

&lt;script&gt;
const title = document.getElementById("page-title");
console.log(title);  // <h1>Welcome</h1>
&lt;/script&gt;
```

Characteristics:

- Returns single element
- Returns `null` if not found [13]
- ID must be unique [8]

getElementsByClassName

Purpose: Selects all elements with specified class [8]

```
const elements = document.getElementsByClassName("highlight");
```

Example:

```
<p>Paragraph 1</p>
<p>Paragraph 2</p>
<div>Div</div>

&lt;script&gt;
const highlighted = document.getElementsByClassName("highlight");
console.log(highlighted.length);  // 3
console.log(highlighted[^0]);      // First <p> element
&lt;/script&gt;
```

Characteristics:

- Returns live HTMLCollection (array-like)
- Access by index: `elements[^0]`
- Updates automatically when DOM changes [8]

getElementsByTagName

Purpose: Selects all elements by tag name [8]

```
const paragraphs = document.getElementsByTagName("p");
```

Example:

```
const allParagraphs = document.getElementsByTagName("p");
console.log(allParagraphs.length);      // Count of </p><p> elements
console.log(allParagraphs[^2].innerHTML); // Content of 3rd </p><p>
```

querySelector

Purpose: Selects first element matching CSS selector [14] [15]

```
const element = document.querySelector(".class-name");
const element = document.querySelector("#id-name");
const element = document.querySelector("div &gt; p");
```

Examples:

```
// By class
const first = document.querySelector(".highlight");

// By ID
const header = document.querySelector("#main-header");

// Complex selectors
const title = document.querySelector('#main h1');
const firstItem = document.querySelector('ul li:first-child');
```

Characteristics:

- Uses CSS selector syntax [14]
- Returns first matching element [15]
- Returns `null` if no match [15]
- More flexible than getElementById [14]

querySelectorAll

Purpose: Selects all elements matching CSS selector [15]

```
const elements = document.querySelectorAll(".highlight");
```

Example:

```
const allHighlighted = document.querySelectorAll(".highlight");

allHighlighted.forEach(element =&gt; {
    element.style.color = "red";
});
```

Characteristics:

- Returns static NodeList (not live) [14]
- Array-like (can use forEach)
- Doesn't update when DOM changes [14]

querySelector vs getElementById

Aspect	getElementById	querySelector
Syntax	`getElementById('id')`	`querySelector('#id')` [14]
Selector Type	ID only	Any CSS selector [14]
Return Type	Element or null	Element or null [14]
Performance	Faster	Slightly slower [14]
Flexibility	Limited	High [14]
Browser Support	Universal	IE8+ [14]

Performance: `getElementById` is faster, but difference is negligible in most applications [14]

Flexibility Example:

```
// getElementById - limited
const elem = document.getElementById('header');

// querySelector - flexible
const elem = document.querySelector('#header');
const elem = document.querySelector('.container &gt; #header');
const elem = document.querySelector('[data-id="123"]');
```

Best Practice: Use `querySelector` for flexibility, `getElementById` for simple ID selections [14]

DOM Manipulation Methods

Changing Content

innerHTML Property:

```
document.getElementById("demo").innerHTML = "New content";
```

textContent Property:

```
document.getElementById("demo").textContent = "New text";
```

Difference:

- `innerHTML`: Interprets HTML tags
- `textContent`: Treats everything as plain text

Changing Attributes

```
document.getElementById("myImage").src = "newImage.jpg";
document.getElementById("myLink").href = "https://example.com";
```

setAttribute Method:

```
element.setAttribute("class", "highlight");
element.setAttribute("data-id", "123");
```

Changing Styles

```
document.getElementById("p1").style.color = "red";
document.getElementById("p1").style.backgroundColor = "lime";
document.getElementById("p2").style.fontFamily = "Arial";
document.getElementById("p2").style.fontSize = "75px";
```

Note: CSS properties with hyphens become camelCase in JavaScript:

- `background-color` → `backgroundColor`
- `font-size` → `fontSize`

- `margin-top` → `marginTop` [8]

Example - Complete DOM Manipulation:

```
&lt;html&gt;
&lt;body&gt;
    </p><h2>Finding HTML Elements by Id</h2>
    <p>Hello world!</p>
    <h1></h1>
    &lt;button onclick="invoke()"&gt;Change&lt;/button&gt;

    &lt;script&gt;
    function invoke() {
        var x = document.getElementById("intro").innerHTML;
        document.getElementById("change").innerHTML = x;
    }
    &lt;/script&gt;
&lt;/body&gt;
&lt;/html&gt;
```

Unit 4.7: JavaScript Event Handling

Events Fundamentals

Definition: Events are actions or occurrences that happen in the browser (clicks, keypresses, page loads, etc.) [8]

Common HTML Events:

Event	When It Fires
`onclick`	Mouse click on element [8]
`ondblclick`	Double click [8]
`onmousemove`	Mouse moves over element [8]
`onmouseover`	Mouse enters element [8]
`onmouseout`	Mouse leaves element [8]
`onkeydown`	Key is pressed [8]
`onkeyup`	Key is released [8]
`onkeypress`	Key is pressed and released [8]
`onload`	Page finishes loading
`onsubmit`	Form is submitted [8]
`onchange`	Input value changes [8]
`onfocus`	Element receives focus [8]
`onblur`	Element loses focus [8]

Event Registration Methods

1. Inline Event Handlers

Syntax: Event attribute in HTML tag

```
&lt;button onclick="alert('Clicked!')"&gt;Click Me&lt;/button&gt;

&lt;input type="button"
       name="myButton"
       value="Click Me"
       onclick="alert('you clicked the button');"&gt;
```

Characteristics:

- Simple and direct [8]
- Mixes HTML and JavaScript (not recommended)
- Difficult to maintain

2. Event Handler Properties

Syntax: Assign function to element's event property

```
const button = document.getElementById("myButton");
button.onclick = function() {
    alert("Button clicked!");
};

// Or with named function
function handleClick() {
    alert("Button clicked!");
}
button.onclick = handleClick;
```

Characteristics:

- Separates HTML and JavaScript
- Only one handler per event type
- Can be overwritten [8]

3. addEventListener (Recommended)

Syntax:

```
element.addEventListener(event, function, useCapture);
```

Parameters:

- **event:** Event name (without "on" prefix)
- **function:** Handler function
- **useCapture:** Boolean for capture phase (optional)

Example:

```
const button = document.getElementById("myButton");

button.addEventListener("click", function() {
    alert("Button clicked!");
});

// Or with named function
function handleClick() {
    alert("Button clicked!");
}
button.addEventListener("click", handleClick);
```

Advantages:

- Multiple handlers for same event [8]
- Better control over event flow
- Can remove listeners
- Modern and recommended approach [8]

Removing Event Listeners:

```
function handleClick() {
    console.log("Clicked!");
}

button.addEventListener("click", handleClick);
button.removeEventListener("click", handleClick);  // Must use same function reference
```

Event Object

Accessing Event Object:

```
element.addEventListener("click", function(event) {
    console.log(event);
});
```

Common Event Properties:

Property	Description
`event.type`	Event type (e.g., "click", "keydown") [8]
`event.target`	Element that triggered event [8]
`event.currentTarget`	Element with event listener attached [8]
`event.clientX` / `event.clientY`	Mouse coordinates [8]
`event.key`	Key pressed (for keyboard events)
`event.preventDefault()`	Prevents default action [8]
`event.stopPropagation()`	Stops event bubbling [8]

Example:

```
document.querySelector("a").addEventListener("click", function(event) {
    event.preventDefault();  // Prevent link navigation
    console.log("Link clicked but not followed");
});
```

Event Delegation

Definition: Attach single event listener to parent element instead of multiple listeners to child elements [8]

Why Use It:

- Performance: Fewer event listeners
- Dynamic elements: Works for elements added later
- Memory efficiency

Example:

```
<ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
</ul>

&lt;script&gt;
// Instead of adding listener to each <li>
document.getElementById("menu").addEventListener("click", function(event) {
    if (event.target.tagName === "LI") {
        console.log("Clicked:", event.target.textContent);
    }
});
&lt;/script&gt;
```

Unit 4.8: JavaScript Form Validation

Client-Side vs Server-Side Validation

Client-Side Validation (JavaScript):

Aspect	Description
When	Before form submission [8]
Speed	Immediate feedback [8]
User Experience	Better (no page reload) [8]
Security	Not secure (can be bypassed) [8]
Purpose	User convenience [8]

Server-Side Validation (Backend):

Aspect	Description
When	After form submission [8]
Speed	Requires server round-trip [8]

Aspect	Description
User Experience	Slower feedback [8]
Security	Secure [8]
Purpose	Data integrity and security [8]

Best Practice: Use both client-side (for UX) and server-side (for security) validation [8]

Validation Flow

```
User fills form
      ↓
JavaScript validation
      ↓
Valid? ──No──→ Show errors, prevent submission
      ↓ Yes
Submit to server
      ↓
Server-side validation
      ↓
Valid? ──No──→ Return errors
      ↓ Yes
Process data
```

Common Validation Checks

What to Validate:

1. **Required fields:** Were they left empty? [8]
2. **Email format:** Valid email address? [8]
3. **Date format:** Valid date? [8]
4. **Data type:** Text in numeric field? Numbers in text field? [8]
5. **Value range:** Number within acceptable range? [8]
6. **Password strength:** Meets requirements?
7. **Matching fields:** Passwords match?

Validation Examples

Preventing Form Submission

```
&lt;form onsubmit="return validateForm()"&gt;
    &lt;input type="text" id="username" name="username"&gt;
    &lt;input type="submit" value="Submit"&gt;
&lt;/form&gt;

&lt;script&gt;
function validateForm() {
    const username = document.getElementById("username").value;

    if (username === "") {
        alert("Username cannot be empty");
        return false;  // Prevent submission
    }
```

```
    return true;  // Allow submission
}
&lt;/script&gt;
```

Email Validation

```
function validateEmail(email) {
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return emailPattern.test(email);
}

function validateForm() {
    const email = document.getElementById("email").value;

    if (!validateEmail(email)) {
        alert("Please enter a valid email address");
        return false;
    }

    return true;
}
```

Number Range Validation

```
function validateAge() {
    const age = document.getElementById("age").value;

    if (isNaN(age)) {
        alert("Age must be a number");
        return false;
    }

    if (age &lt; 18 || age &gt; 100) {
        alert("Age must be between 18 and 100");
        return false;
    }

    return true;
}
```

Password Matching

```
function validatePasswords() {
    const password = document.getElementById("password").value;
    const confirmPassword = document.getElementById("confirmPassword").value;

    if (password !== confirmPassword) {
        alert("Passwords do not match");
        return false;
    }

    if (password.length &lt; 8) {
        alert("Password must be at least 8 characters");
        return false;
    }
```

```
    return true;
}
```

Complete Validation Example

```
&lt;html&gt;
&lt;head&gt;
    &lt;title&gt;Form Validation&lt;/title&gt;
&lt;/head&gt;
&lt;body&gt;
    &lt;form onsubmit="return validateForm()"&gt;
        &lt;label for="name"&gt;Name:&lt;/label&gt;
        &lt;input type="text" id="name" name="name"&gt;
        <span></span>
        <br>

        &lt;label for="email"&gt;Email:&lt;/label&gt;
        &lt;input type="email" id="email" name="email"&gt;
        <span></span>
        <br>

        &lt;label for="age"&gt;Age:&lt;/label&gt;
        &lt;input type="number" id="age" name="age"&gt;
        <span></span>
        <br>

        &lt;input type="submit" value="Submit"&gt;
    &lt;/form&gt;

    &lt;script&gt;
    function validateForm() {
        let isValid = true;

        // Clear previous errors
        document.getElementById("nameError").textContent = "";
        document.getElementById("emailError").textContent = "";
        document.getElementById("ageError").textContent = "";

        // Validate name
        const name = document.getElementById("name").value.trim();
        if (name === "") {
            document.getElementById("nameError").textContent = "Name is required";
            isValid = false;
        }

        // Validate email
        const email = document.getElementById("email").value.trim();
        const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (!emailPattern.test(email)) {
            document.getElementById("emailError").textContent = "Invalid email format";
            isValid = false;
        }

        // Validate age
        const age = document.getElementById("age").value;
        if (age &lt; 18 || age &gt; 100) {
            document.getElementById("ageError").textContent = "Age must be between 18 and 100";
            isValid = false;
        }
```

```
        return isValid;
    }
    &lt;/script&gt;
&lt;/body&gt;
&lt;/html&gt;
```

Module 5: Bootstrap Framework (CO4)

Bootstrap Fundamentals

Introduction to Bootstrap

Definition: Bootstrap is a free, open-source CSS framework for developing responsive, mobile-first websites [16] [17].

Key Features:

- **12-column grid system:** Flexible layout structure [16] [18]
- **Pre-built components:** Buttons, navbars, modals, forms [16]
- **Responsive utilities:** Show/hide elements by screen size [16]
- **Mobile-first approach:** Optimized for small screens first [16]

Advantages:

- Faster development with pre-built components [16]
- Consistent design across browsers
- Responsive by default
- Extensive documentation and community support [16]

Bootstrap Grid System

Core Concept: Page divided into 12 equal-width columns that adapt to screen sizes [16] [17] [18]

Grid Structure:

```
Container
  └── Row
      ├── Column (col-*)
      ├── Column (col-*)
      └── Column (col-*)
```

Components:

1. **Container:** Wraps content and centers it [19]
   - `.container`: Fixed-width
   - `.container-fluid`: Full-width
2. **Row:** Horizontal group of columns [19]
   - Use `.row` class
   - Columns must be inside rows
3. **Columns:** Content holders [19]

- Use `.col-*` classes
- Must add up to 12 per row

Responsive Breakpoints

Bootstrap Breakpoint Classes:

Class Prefix	Screen Size	Container Width	Device Type
`.col-`	<576px	100%	Extra small (phones portrait) [19]
`.col-sm-`	≥576px	540px	Small (phones landscape) [16] [18]
`.col-md-`	≥768px	720px	Medium (tablets) [16] [18]
`.col-lg-`	≥992px	960px	Large (desktops) [16] [18]
`.col-xl-`	≥1200px	1140px	Extra large (large desktops) [16] [18]

How Breakpoints Work:

Classes apply to that breakpoint **and all larger sizes** [17] [18]

Example:

```
<div>Column</div>
```

Effect:

- **xs (<576px):** Full width (12 columns)
- **sm+ (≥576px):** 6 columns (50% width)

Grid Examples

Equal Columns

```
<div>
    <div>
        <div>Column 1</div>
        <div>Column 2</div>
        <div>Column 3</div>
    </div>
</div>
```

Result: Three equal-width columns

Specific Column Widths

```
<div>
    <div>
        <div>4 columns</div>
        <div>8 columns</div>
    </div>
</div>
```

Result: 33.33% and 66.67% width columns

Responsive Columns

```
<div>
    <div>
        <div>
            Responsive Column
        </div>
    </div>
</div>
```

Behavior:

Screen Size	Column Width
xs (<576px)	100% (full width) [16]
sm (≥576px)	50% (6/12 columns) [16]
md (≥768px)	33.33% (4/12 columns) [16]
lg (≥992px)	25% (3/12 columns) [16]

Multi-Row Example

```
<div>
    <div>
        <div>Column 1</div>
        <div>Column 2</div>
        <div>Column 3</div>
        <div>Column 4</div>
    </div>
</div>
```

Responsive Behavior:

- **Mobile (xs):** 1 column per row (stacked)
- **Small (sm):** 2 columns per row
- **Medium (md):** 3 columns per row
- **Large (lg+):** 4 columns per row [16]

Bootstrap Container Types

Fixed-Width Container

```
<div>

</div>
```

Behavior: Centers content with fixed max-width based on breakpoint [1] [19]

Fluid Container

```
<div>

</div>
```

Behavior: Full width (100%) at all breakpoints [1] [19]

Comparison:

Aspect	`.container`	`.container-fluid`
Width	Fixed max-width	Full width (100%) [1]
Responsive	Yes (changes at breakpoints)	Yes (always 100%) [1]
Use Case	Standard layouts	Full-width designs [1]

Bootstrap Best Practices

Responsive Design:

1. **Mobile-First Approach:** Design for smallest screens first, enhance for larger [16] [20]
2. **Use Three or More Breakpoints:** Cover mobile, tablet, desktop [21]
3. **Content Prioritization:** Show essential content on small screens [21]
4. **Minimize Content:** Avoid clutter on mobile devices [21]
5. **Test Across Devices:** Ensure proper rendering on all screen sizes [16]

Grid System Usage:

1. **Master 12-Column System:** Understand how columns divide [16]
2. **Balanced Content:** Distribute content evenly across columns [16]
3. **Avoid Excessive Nesting:** Keep grid structure simple
4. **Use Responsive Classes:** Apply appropriate breakpoint classes [16]

Performance Optimization:

1. **Optimize Images:** Use responsive images (`img-fluid` class) [20]
2. **Leverage Utility Classes:** Use Bootstrap utilities instead of custom CSS [16]
3. **Minimize HTTP Requests:** Combine files where possible [20]
4. **Use CDN:** Load Bootstrap from CDN for faster delivery

Accessibility:

1. **Follow WCAG Guidelines:** Ensure accessible design [16] [20]
2. **Use ARIA Attributes:** Improve screen reader support [16]
3. **Keyboard Navigation:** Ensure all features accessible via keyboard [20]
4. **Color Contrast:** Maintain sufficient contrast ratios [20]

Quick Review Summary Tables

HTML Quick Reference

Essential HTML Elements

Element	Purpose	Example
<html>	Root element	<html>...</html>
<head>	Document metadata	<head><title>...</title></head>
<body>	Visible content	<body>...</body>
<h1>-</h1> <h6>	Headings	</h6><h1>Title</h1>
	Paragraph	Text
<a>	Hyperlink	Link
<img>	Image	<img>
	Division/container	...
	Inline container	...
/	Lists	Item
<table>	Table	<table><tr><td>...</td></tr></table>
<form>	Form	<form>...</form>

HTML Form Input Types

Type	Purpose	Example
text	Single-line text	<input type="text">
password	Password (masked)	<input type="password">
email	Email address	<input type="email">
number	Numeric input	<input type="number" min="1" max="10">
date	Date picker	<input type="date">
checkbox	Checkbox	<input type="checkbox">
radio	Radio button	<input type="radio" name="group">
submit	Submit button	<input type="submit">
reset	Reset button	<input type="reset">

CSS Quick Reference

CSS Selectors Cheat Sheet

Selector	Example	Selects
Element	`p { }`	All elements
Class	`.highlight { }`	Elements with class="highlight"
ID	`#header { }`	Element with id="header"
Universal	`* { }`	All elements
Descendant	`div p { }`	All inside (any level)
Child	`div > p { }`	directly inside
Multiple	`h1, h2, h3 { }`	All h1, h2, h3 elements
Pseudo-class	`a:hover { }`	Link when hovered
Pseudo-element	`p::first-line { }`	First line of paragraph

CSS Box Model Properties

Property	Purpose	Example
`width`	Element width	`width: 500px;`
`height`	Element height	`height: 300px;`
`padding`	Space inside border	`padding: 10px;`
`border`	Border around element	`border: 2px solid black;`
`margin`	Space outside border	`margin: 20px;`
`box-sizing`	Box model calculation	`box-sizing: border-box;`

Common CSS Properties

Category	Property	Example
Text	`color`	`color: red;`
	`font-family`	`font-family: Arial;`
	`font-size`	`font-size: 16px;`
	`text-align`	`text-align: center;`
Background	`background-color`	`background-color: blue;`
	`background-image`	`background-image: url('img.jpg');`
Layout	`display`	`display: block;`
	`position`	`position: relative;`
	`float`	`float: left;`

JavaScript Quick Reference

JavaScript Data Types

Type	Example	Typeof Result
Number	`var x = 42;`	`"number"`
String	`var s = "hello";`	`"string"`
Boolean	`var b = true;`	`"boolean"`
Undefined	`var u;`	`"undefined"`
Null	`var n = null;`	`"object"`
Object	`var o = {};`	`"object"`
Array	`var a = [];`	`"object"`

JavaScript Operators

Category	Operators	Example
Arithmetic	`+`, `-`, `*`, `/`, `%`, `++`, `--`	`5 + 3 = 8`
Comparison	`==`, `!=`, `===`, `!==`, `>`, `<`, `>=`, `<=`	`5 === "5"` → `false`
Logical	`&&`, `||`, `!`	`true && false` → `false`
Assignment	`=`, `+=`, `-=`, `*=`, `/=`	`x += 5`

JavaScript DOM Methods

Method	Purpose	Returns
`getElementById(id)`	Select by ID	Single element
`getElementsByClassName(class)`	Select by class	HTMLCollection
`getElementsByTagName(tag)`	Select by tag	HTMLCollection
`querySelector(selector)`	Select first match (CSS selector)	Single element
`querySelectorAll(selector)`	Select all matches (CSS selector)	NodeList

JavaScript Event Types

Event	When Triggered	Example
`click`	Element clicked	`element.onclick = func;`
`dblclick`	Element double-clicked	`element.ondblclick = func;`
`mouseover`	Mouse enters element	`element.onmouseover = func;`
`mouseout`	Mouse leaves element	`element.onmouseout = func;`
`keydown`	Key pressed	`element.onkeydown = func;`
`keyup`	Key released	`element.onkeyup = func;`
`submit`	Form submitted	`form.onsubmit = func;`

Event	When Triggered	Example
`change`	Input value changed	`input.onchange = func;`
`load`	Page finished loading	`window.onload = func;`

Bootstrap Quick Reference

Bootstrap Grid Classes

Class	Breakpoint	Screen Width	Container Width
`.col-`	Extra small	<576px	100%
`.col-sm-`	Small	≥576px	540px
`.col-md-`	Medium	≥768px	720px
`.col-lg-`	Large	≥992px	960px
`.col-xl-`	Extra large	≥1200px	1140px

Bootstrap Container Classes

Class	Behavior
`.container`	Responsive fixed-width container
`.container-fluid`	Full-width container (100%)

Top 10 Most Testable Facts Per Module

Module 1: Internet & HTML

1. **URL consists of 4 parts:** Protocol, domain name, pathname, filename [2]
2. **IPv4 format:** 4 octets, 32-bit, range 0-255 per field [2]
3. **IPv6 format:** 8 fields, 128-bit, hexadecimal [2]
4. **HTTP is stateless:** Each request is independent [2]
5. **DNS resolves:** Domain names to IP addresses [2]
6. **Common HTTP methods:** GET (retrieve), POST (send data), PUT (update), DELETE (remove) [2]
7. **Block elements:** Start on new line, take full width [2]
8. **Inline elements:** Same line, necessary width only [2]
9. **Semantic elements clearly describe content:** `<article>`, `<section>`, `<nav>`, `<header>`, `<footer>` [2]
10. **Form GET vs POST:** GET visible in URL/limited, POST hidden/unlimited [2]

Module 2: CSS Fundamentals

1. **CSS specificity order:** Inline > ID > Class > Element [5]
2. **Cascading order:** Browser default < External < Internal < Inline [5]
3. **Three CSS types:** Inline, Internal, External [5]

4. **Box model layers (inside-out):** Content, Padding, Border, Margin [5] [6]
5. **Margins collapse vertically:** Larger margin wins [6]
6. **Child selector (`>`):** Direct children only [5]
7. **Descendant selector (space):** Any level descendants [5]
8. **Pseudo-classes define special states:** `:hover`, `:active`, `:focus` [5]
9. **Em vs Rem:** Em relative to parent, Rem relative to root [6]
10. **Position values:** Static, Relative, Fixed, Absolute, Sticky [6]

Module 3: Advanced CSS

1. **Link states order:** `:link`, `:visited`, `:hover`, `:active` (LoVe HAte) [6]
2. **Relative positioning:** Relative to normal position, space reserved [6]
3. **Absolute positioning:** Relative to positioned ancestor, removed from flow [6]
4. **Fixed positioning:** Relative to viewport, stays when scrolling [6]
5. **Float property:** Creates flexible layouts, requires clear [6]
6. **Clear property values:** Left, right, both, none [6]
7. **Opacity range:** 0.0 (transparent) to 1.0 (opaque) [6]
8. **Responsive image:** `max-width: 100%; height: auto;` [6]
9. **Center image:** `display: block; margin: 0 auto;` [6]
10. **Table hover effect:** `tr:hover { background-color: gray; }` [6]

Module 4: JavaScript

1. **JavaScript is dynamically typed:** No type declarations needed [8]
2. **Three variable declarations:** `var` (function scope), `let` (block scope), `const` (constant) [9] [10]
3. `==` **vs** `===`**:** `==` with type conversion, `===` strict (no conversion) [8]
4. **Primitive types:** Number, String, Boolean, Null, Undefined, Symbol [8]
5. `typeof null` **returns:** `"object"` (JavaScript quirk) [8]
6. **Function scope:** Inner functions access outer variables, not vice versa [8]
7. **Closures:** Functions remembering variables from parent scope [11] [12]
8. **DOM stands for:** Document Object Model [8]
9. `getElementById` **returns:** Single element or null [8] [13]
10. `querySelectorAll` **returns:** Static NodeList (not live) [14] [15]

Module 5: Bootstrap

1. **Bootstrap grid:** 12-column system [16] [17] [18]
2. **Grid structure:** Container > Row > Columns [19]
3. **Mobile-first approach:** Design for small screens first, enhance for larger [16] [20]
4. **Five breakpoints:** xs (<576px), sm (≥576px), md (≥768px), lg (≥992px), xl (≥1200px) [16] [18] [19]
5. `.container` **vs** `.container-fluid`**:** Fixed-width vs full-width [1] [19]
6. **Breakpoint classes apply to:** That breakpoint and all larger sizes [17] [18]

7. **Column classes must add to:** 12 per row [22]
8. **Responsive column example:** `col-sm-6 col-md-4 col-lg-3` [16]
9. **Bootstrap benefits:** Faster development, consistent design, responsive by default [16]
10. **Responsive image class:** `img-fluid` [20]

Common Question Patterns with Answers

Pattern 1: Compare and Contrast

Q: Compare IPv4 and IPv6.

Answer:

- **IPv4:** 32-bit, 4 octets, dot-separated, range 0-255 (e.g., 192.168.2.33)
- **IPv6:** 128-bit, 8 fields, colon-separated, hexadecimal (e.g., FDEC:BA98:7654:3210:ADEC:BDFF:2990:FFFF) [2]

Q: Compare GET and POST methods.

Answer:

Aspect	GET	POST
Visibility	Data in URL	Data in request body
Security	Not secure	More secure
Data Length	Limited (~3000 chars)	Unlimited
Use Case	Non-sensitive data	Sensitive/large data [2]

Q: Compare `getElementById` and `querySelector`.

Answer:

- `getElementById`: Selects by ID only, faster, returns single element
- `querySelector`: Uses CSS selectors, more flexible, returns first match [14] [15]

Pattern 2: Explain Process/How It Works

Q: Explain how DNS resolution works.

Answer:

1. User enters domain name (e.g., www.example.com)
2. Browser sends DNS query to DNS server
3. DNS server responds with IP address
4. Browser uses IP to connect to web server [2]

Q: Explain the CSS cascade.

Answer:
When styles conflict, CSS resolves using:

1. **Origin:** Inline > Internal > External > Browser default

2. **Specificity:** ID > Class > Element
3. **Source order:** Later declarations win [5]

Q: Explain how JavaScript closures work.

Answer:

1. Outer function creates variable
2. Inner function accesses that variable
3. Outer function returns inner function
4. Inner function retains access to outer variable even after outer function finishes [11] [12]

Pattern 3: List and Explain

Q: List HTTP methods and their purposes.

Answer:

- **GET:** Retrieve data (requesting web page)
- **POST:** Send data to create/update (submitting form)
- **PUT:** Update existing resource
- **DELETE:** Remove resource
- **HEAD:** Retrieve headers only
- **OPTIONS:** Retrieve communication options [2]

Q: List CSS position values and explain each.

Answer:

- **Static:** Default, normal flow
- **Relative:** Relative to normal position, space reserved
- **Fixed:** Fixed to viewport, stays when scrolling
- **Absolute:** Relative to positioned ancestor, removed from flow
- **Sticky:** Switches between relative and fixed [6]

Pattern 4: Write Code

Q: Write HTML form with validation.

Answer:

```
&lt;form onsubmit="return validateForm()"&gt;
    &lt;label&gt;Name: &lt;input type="text" id="name" required&gt;&lt;/label&gt;
    &lt;input type="submit"&gt;
&lt;/form&gt;

&lt;script&gt;
function validateForm() {
    var name = document.getElementById("name").value;
    if (name === "") {
        alert("Name required");
        return false;
    }
```

```
    return true;
}
&lt;/script&gt;
```

Q: Write CSS to center a div.

Answer:

```
div {
    width: 500px;
    margin: 0 auto;  /* Horizontal centering */
}
```

Q: Write JavaScript to change element color on click.

Answer:

```
document.getElementById("myButton").addEventListener("click", function() {
    this.style.color = "red";
});
```

Pattern 5: Identify Errors

Q: What's wrong with this code?

```
<h1>This is my heading.</h1>

&lt;style&gt;
h1 { color: blue; }
.main-heading { color: red; }
&lt;/style&gt;
```

Answer: Nothing is wrong. The heading will be **red** because class selector has higher specificity than element selector [5].

Q: What's the issue?

```
var v = ("5" === 5);
```

Answer: `v` will be `false` because `===` performs strict comparison without type conversion. String "5" is not strictly equal to number 5 [8].

Mini Quiz: Mixed Questions

Multiple Choice Questions

1. Which HTTP status code indicates success?
a) 404
b) 500
c) 200
d) 300

Answer: c) 200 [2]

2. What does CSS stand for?
a) Computer Style Sheets
b) Cascading Style Sheets
c) Creative Style Sheets
d) Colorful Style Sheets

Answer: b) Cascading Style Sheets [5]

3. Which selector has the highest specificity?
a) Element selector
b) Class selector
c) ID selector
d) Universal selector

Answer: c) ID selector [5]

4. What does DOM stand for?
a) Document Object Model
b) Data Object Model
c) Digital Object Model
d) Dynamic Object Model

Answer: a) Document Object Model [8]

5. Bootstrap grid system is based on how many columns?
a) 10
b) 12
c) 16
d) 24

Answer: b) 12 [16] [17] [18]

6. Which JavaScript keyword declares a constant?
a) var
b) let
c) const
d) constant

Answer: c) const [9] [10]

7. Which CSS property is used for rounded corners?
a) corner-radius
b) border-curve
c) border-radius
d) round-border

Answer: c) border-radius [6]

8. What is the correct HTML for creating a hyperlink?
a) `<a>Link</a>`
b) `<a href="http://example.com">Link</a>`
c) `&lt;link href="http://example.com"&gt;Link&lt;/link&gt;`
d) `&lt;url&gt;http://example.com&lt;/url&gt;`

Answer: b) `<a href="http://example.com">Link</a>` [2]

9. Which method removes an element from the DOM?
a) `delete()`
b) `remove()`
c) `removeChild()`
d) `deleteElement()`

Answer: c) `removeChild()`

10. What does the `!==` operator do in JavaScript?
a) Not equal with type conversion
b) Strict not equal
c) Assignment
d) Comparison

Answer: b) Strict not equal [8]

Short Answer Questions

1. Explain the difference between `<div>` and `<span>`.

Answer: `<div>` is a block-level element (starts on new line, takes full width), while `<span>` is inline (stays in line, takes necessary width only) [2].

2. What is the purpose of the `alt` attribute in `<img>` tags?

Answer: Provides alternative text description for accessibility (screen readers) and displays if image fails to load [2].

3. Explain margin collapse in CSS.

Answer: When vertical margins of adjacent elements touch, they collapse into a single margin (the larger one wins). Horizontal margins do not collapse [6].

4. What is the difference between `let` and `const`?

Answer: `let` allows reassignment of values, while `const` creates a constant that cannot be reassigned [9] [10].

5. What does `event.preventDefault()` do?

Answer: Prevents the default action associated with an event (e.g., prevents form submission or link navigation) [8].

Study Tips and Exam Strategies

Preparation Strategy

Week-by-Week Study Plan

Week 1-2: HTML & Internet Fundamentals (Module 1)

- Review all HTML tags and attributes
- Practice creating forms with validation
- Memorize URL structure and HTTP methods
- Create sample web pages

Week 3: CSS Fundamentals (Module 2)

- Master CSS selectors and specificity
- Practice box model calculations
- Learn all CSS properties
- Style sample HTML pages

Week 4: Advanced CSS (Module 3)

- Study positioning techniques
- Practice layout creation
- Review responsive design concepts
- Create complete page layouts

Week 5: JavaScript (Module 4)

- Review data types and operators
- Practice DOM manipulation
- Study event handling thoroughly
- Write validation scripts

Week 6: Bootstrap & Review (Module 5 + Revision)

- Learn Bootstrap grid system
- Practice responsive layouts
- Review all modules
- Practice exam questions

Exam Tips

For Multiple Choice Questions

1. **Eliminate obviously wrong answers first**
2. **Look for keywords:** "always," "never," "all," "none" (often incorrect)
3. **HTML/CSS syntax questions:** Pay attention to spelling and symbols
4. **JavaScript comparisons:** Remember `==` vs `===` differences [8]
5. **Box model calculations:** Calculate step by step [7]

For Coding Questions

1. **Read requirements carefully**
2. **Plan before coding:** Sketch structure
3. **Check syntax:** Closing tags, semicolons, brackets
4. **Test logic mentally:** Walk through code
5. **Include comments:** Show understanding

For Descriptive Questions

1. **Use proper technical terminology**
2. **Provide examples when possible**
3. **Structure answers:** Introduction, explanation, conclusion
4. **Draw diagrams:** Box model, DOM tree, layouts
5. **Compare/contrast:** Use tables for clarity

Common Mistakes to Avoid

HTML

- Forgetting closing tags
- Mixing up attributes (`href` vs `src`)
- Invalid nesting of elements
- Not using semantic elements appropriately [3] [4]

CSS

- Forgetting semicolons
- Wrong specificity calculations
- Box model miscalculations
- Confusing margin and padding [5] [6]

JavaScript

- Using `=` instead of `==` or `===`
- Scope errors (accessing variables outside scope)
- Forgetting `return` in functions
- Not preventing form submission when needed [8]

Bootstrap

- Not wrapping columns in rows
- Columns not adding to 12
- Wrong breakpoint classes
- Not using containers properly [16] [19]

Memory Aids

Mnemonics

HTTP Methods: Get **P**ost **P**ut **D**elete → "**G**ood **P**rogrammers **P**ractice **D**aily" [2]

Link States: Link **V**isited **H**over **A**ctive → "**L**o**V**e **HA**te" [6]

Box Model (Inside-Out): Content **P**adding **B**order **M**argin → "**C**ats **P**refer **B**ig **M**eal" [5] [6]

CSS Specificity: **I**nline > **I**D > **C**lass > **E**lement → "**I** **I**ncreasingly **C**rave **E**spresso" [5]

Bootstrap Breakpoints: **X**S **S**M **M**D **L**G **X**L → E**x**tra **S**mall, **S**mall, **M**edium, **L**arge, E**x**tra **L**arge [16] [19]

Visual Memory

Draw Box Model:

```
┌──────────── Margin ────────────┐
│  ┌──────── Border ──────┐     │
│  │  ┌── Padding ──┐    │     │
│  │  │   Content   │    │     │
│  │  └─────────────┘    │     │
│  └─────────────────────┘     │
└──────────────────────────────┘
```

HTML Document Structure:

```
&lt;html&gt;
  &lt;head&gt;
    &lt;title&gt;Page Title&lt;/title&gt;
  &lt;/head&gt;
  &lt;body&gt;

  &lt;/body&gt;
&lt;/html&gt;
```

JavaScript Event Flow:

```
User Action → Event Triggered → Handler Executed → Result
```

References and Citations

[1] WFQN2.pdf - RV University Exam Question Paper
[5] WFUNIT2.pdf - Cascading Style Sheets
[6] WFUNIT3.pdf - CSS Advanced Topics
[8] WFUNIT4.pdf - JavaScript Fundamentals
[2] WFUNIT1.pdf - HTML and Internet Fundamentals
[16] BrowserStack - Bootstrap Responsive Design
[17] Bootstrap Official Documentation - Grid System
[18] Bootstrap v5.0 Grid System Documentation
[22] W3Schools - Bootstrap Grid System
[19] GeeksforGeeks - Bootstrap Grid System
[23] UXtweak - UX Design Principles
[3] GeeksforGeeks - HTML5 Semantics
[4] Semrush - Semantic HTML Guide
[11] Alooba - JavaScript Closures
[14] Stack Overflow - querySelector vs getElementById
[24] MDN - CSS Box Model
[12] MDN - JavaScript Closures
[13] freeCodeCamp - DOM Manipulation

[7] W3Schools - CSS Box Model
[25] Joel Olawanle - JavaScript Closures
[15] JavaScript.info - DOM Selection Methods
[20] BrowserStack - Web Design Best Practices 2025
[21] Interaction Design Foundation - Responsive Design
[9] Scribbler - JavaScript ES6 Features
[10] W3Schools - ES6 Features

END OF STUDY NOTES

These comprehensive notes cover all essential topics for CS1307 - Web Fundamentals and UX Design. Use this as your primary study resource alongside hands-on practice. Good luck with your exam preparation!
[26] [27] [28] [29] [30] [31] [32] [33] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] [51] [52] [53] [54] [55] [56] [57] [58] [59] [60] [61] [62] [63] [64] [65] [66] [67] [68] [69] [70] [71] [72] [73] [74] [75] [76] [77] [78] [79] [80] [81] [82]

⁂

1. WFQN2.pdf
2. WFUNIT1.pdf
3. https://www.geeksforgeeks.org/html/html5-semantics/
4. https://www.semrush.com/blog/semantic-html5-guide/
5. WFUNIT2.pdf
6. WFUNIT3.pdf
7. https://www.w3schools.com/css/css_boxmodel.asp
8. WFUNIT4.pdf
9. https://fullscale.io/blog/essential-web-technologies/
10. https://www.geeksforgeeks.org/blogs/web-development-best-practices/
11. https://www.alooba.com/skills/concepts/javascript-49/javascript-closure/
12. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures
13. https://www.freecodecamp.org/news/the-javascript-dom-manipulation-handbook/
14. https://stackoverflow.com/questions/26848289/queryselector-vs-getelementbyid
15. https://javascript.info/searching-elements-dom
16. https://www.browserstack.com/guide/bootstrap-mobile-responsive
17. https://getbootstrap.com/docs/4.0/layout/grid/
18. https://getbootstrap.com/docs/5.0/layout/grid/
19. https://www.geeksforgeeks.org/bootstrap/bootstrap-grid-system/
20. https://www.interaction-design.org/literature/article/responsive-design-let-the-device-do-the-work
21. https://scribbler.live/2024/04/12/ES6-Features-with-Examples.html
22. https://www.w3schools.com/bootstrap/bootstrap_grid_system.asp
23. https://www.uxtweak.com/ux-design/principles/
24. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Box_model
25. https://joelolawanle.com/blog/javascript-closures
26. http://arxiv.org/pdf/2408.11140.pdf
27. https://arxiv.org/pdf/2001.02921.pdf
28. https://arxiv.org/html/2404.13521v1
29. https://arxiv.org/ftp/arxiv/papers/2311/2311.16601.pdf

30. https://arxiv.org/pdf/2401.16139.pdf
31. https://codefinity.com/courses/v2/4333ab1f-0a72-431c-b9d0-73a7592739d4/d1bf88df-105d-4011-907b-24ab50ccef81/495bb450-0278-4f38-91ab-5c3db60a1501
32. https://thecuneiform.com/insights/the-fundamentals-of-ui-and-ux-design-a-comprehensive-guide/
33. https://designmodo.com/bootstrap-5-layout/
34. https://uxplaybook.org/articles/7-ux-fundamentals-a-comprehensive-guide
35. https://www.dreamhost.com/blog/html5-semantic/
36. https://getbootstrap.com/docs/3.4/css/
37. https://www.lyssna.com/blog/ux-design-principles/
38. https://www.saffronedge.com/blog/semantic-html/
39. https://getbootstrap.com
40. https://trymata.com/blog/what-is-user-experience-ux-design/
41. https://www.dhiwise.com/post/exploring-the-significance-of-semantic-html-elements
42. https://www.packtpub.com/en-us/learning/how-to-tutorials/bootstrap-grid-system-responsive-website
43. https://www.reva.edu.in/naac2023/Extended_Profile/3.1/Handbook/B_Tech_CSE_2021_2025.pdf
44. https://iisc.ac.in/wp-content/uploads/2025/07/SoI-August-2025-Term.pdf
45. https://intranet.cb.amrita.edu/sites/default/files/2023-2024/Sylabus/BTech-CSE-CURRICULUM&SYLLABUS-2023.pdf
46. https://www.nielit.gov.in/sites/default/files/revised_b_level_0.pdf
47. https://mrce.in/ebooks/Computer Systems A Programmer’s Perspective 3rd Ed.pdf
48. https://www.aitpune.com/Documents/Comp/TE_Computer_2019_Course_revised_draft_7June2021.pdf
49. https://www.mewaruniversity.org/uploads/files/csepro(1).pdf
50. https://www.lpude.in/SLMs/Master of Computer Applications/Sem_2/DECAP456_INTRODUCTION_TO_BIG_DATA.pdf
51. https://arxiv.org/pdf/1802.02974.pdf
52. https://arxiv.org/pdf/0912.2861.pdf
53. https://arxiv.org/pdf/2302.00238.pdf
54. https://arxiv.org/html/2407.06924v1
55. https://arxiv.org/pdf/2108.07389.pdf
56. http://arxiv.org/pdf/1510.00925.pdf
57. http://arxiv.org/pdf/1504.08110.pdf
58. WFQN1.pdf
59. https://www.tandfonline.com/doi/full/10.1080/23311835.2016.1247505
60. https://www.geeksforgeeks.org/css/css-box-model/
61. https://www.w3schools.com/js/js_function_closures.asp
62. https://dev.to/eidorianavi/queryselector-vs-getelementbyid-gm1
63. https://www.youtube.com/watch?v=qhiQGPtD1PQ
64. https://online-journals.org/index.php/i-jet/article/download/2916/2882
65. https://arxiv.org/html/2502.15708v1
66. https://arxiv.org/pdf/0801.2618.pdf
67. https://arxiv.org/pdf/2309.00744.pdf
68. https://arxiv.org/pdf/2312.02992.pdf
69. https://wjaets.com/sites/default/files/WJAETS-2024-0051.pdf
70. https://shareok.org/bitstream/11244/25434/1/10.1177.154193120304701109.pdf
71. https://www.tandfonline.com/doi/full/10.1080/23744731.2024.2444819

72. https://www.mdpi.com/2078-2489/10/10/314/pdf
73. https://www.browserstack.com/guide/website-design-tips
74. https://www.designveloper.com/blog/responsive-web-design-principles/
75. https://www.w3schools.com/nodejs/nodejs_es6.asp
76. https://thestory.is/en/journal/responsive-web-design/
77. https://www.w3schools.com/js/js_es6.asp
78. http://arxiv.org/pdf/1511.07042.pdf
79. https://javascript.plainenglish.io/everything-i-use-daily-as-a-web-developer-in-2025-06438a58dbb1
80. https://www.andacademy.com/resources/blog/ui-ux-design/what-is-responsive-design/
81. https://www.boardinfinity.com/blog/top-10-features-of-es6/ </code>
</h6> </code></blockquote>
</h6>
82. https://arxiv.org/html/2409.01339v1